

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



TOÁN RỜI RẠC VÀ THUẬT TOÁN
BÀI THU HOẠCH ĐÁNH GIÁ HẾT MÔN

Nhóm 4

Nguyễn Thị Nhật Linh - 25007871

Lê Thị Liên - 25007874

Trần Tùng Lâm - 25007944

Nguyễn Phúc Lợi - 25007868

Mục lục

PHẦN 1 - TỔNG QUAN MÔN HỌC: TÓM LƯỢC CÁC NỘI DUNG ĐÃ LĨNH HỘI ĐƯỢC TỪ MÔN HỌC (VỀ LÝ THUYẾT, THUẬT TOÁN VÀ ỨNG DỤNG).....	5
1. PHÂN TÍCH THUẬT TOÁN.....	5
1.1. Khái niệm và vai trò.....	5
1.2. Mục tiêu của phân tích thuật toán.....	5
1.3. Chứng minh tính đúng của thuật toán.....	5
1.3.1. Thuật toán đệ quy dùng quy nạp toán học.....	5
1.3.2. Thuật toán không đệ quy dung bất biến vòng lặp.....	6
1.4. Phân tích độ phức tạp của thuật toán.....	6
1.5. Phân loại bài toán P và NP (độ phức tạp P, NP).....	7
2. MỘT SỐ PHƯƠNG PHÁP THIẾT KẾ THUẬT TOÁN CƠ BẢN.....	8
2.1. Các phương pháp chính xác.....	8
2.1.1. Kỹ thuật đệ quy.....	8
2.1.2. Phương pháp chia để trị và quy hoạch động (phương pháp phân rã bài toán).....	8
2.1.3. Phương pháp Quay lui và Nhánh cận (nhóm tìm kiếm không gian nghiệm).....	10
2.2. Các phương pháp có sự thỏa hiệp.....	11
3. LÝ THUYẾT ĐỒ THỊ VÀ ỨNG DỤNG.....	14
3.1. Nhận thức và hiểu biết về đồ thị.....	14
3.1.1. Khái niệm cơ bản về đồ thị.....	14
3.1.2. Cấu trúc dữ liệu biểu diễn đồ thị.....	15
3.1.3. Mô hình và ứng dụng của đồ thị trong thực tiễn.....	15
3.2. Đường đi trong đồ thị.....	16
3.2.1. Khái niệm.....	16
3.2.2. Các thuật toán cơ bản.....	16
3.2.3. Các bước thực hiện.....	18
3.2.4. Ứng dụng trong thực tế.....	18
3.3. Đường đi ngắn nhất trong đồ thị.....	19
3.3.1. Phát biểu bài toán.....	19
3.3.2. Các thuật toán chính.....	19
3.3.3. Cách giải bài toán.....	20
3.3.4. Ứng dụng trong thực tế.....	20

3.4. Chu trình trong đồ thị.....	21
3.4.1. Khái niệm.....	21
3.4.2. Phát hiện chu trình.....	21
3.4.3. Chu trình Euler.....	22
3.4.4. Chu trình Hamilton.....	23
4. CÂY VÀ ỨNG DỤNG.....	24
4.1. Khái Niệm Cây (Tree).....	24
4.1.1. Định nghĩa.....	24
4.1.2. Cây Có Gốc (Rooted Tree).....	24
4.2. Cây Nhị Phân (Binary Tree).....	24
4.2.1. Định Nghĩa.....	24
4.2.2. Các Loại Cây Nhị Phân.....	24
4.2.3. Cài Đặt Cây Nhị Phân.....	25
4.3. Thuật toán duyệt cây (Tree Traversal).....	25
4.3.1. Duyệt Thứ Tự Trước (Pre-order Traversal).....	26
4.3.2. Duyệt Thứ Tự Sau (Post-order Traversal).....	26
4.3.3. Duyệt Thứ Tự Giữa (In-order Traversal).....	26
4.3.4. So Sánh Ba Phép Duyệt.....	26
4.4. Ứng Dụng Của Cây Nhị Phân.....	27
4.4.1. Cây Biểu Thức (Arithmetic Expression Trees).....	27
4.4.2. Cây Tìm Kiếm Nhị Phân (Binary Search Tree - BST).....	27
4.4.3. Sắp Xếp Vun Đống (Heap Sort).....	27
4.5. Cây Bao Trùm Tối Thiểu (Minimum Spanning Tree - MST).....	27
4.5.1. Định Nghĩa.....	27
4.5.2. Thuật Toán Prim.....	27
4.5.3. Thuật Toán Kruskal.....	28
4.5.4. So Sánh Prim và Kruskal.....	28
4.6. Ứng Dụng Thực Tiễn.....	28
4.6.1. Ứng Dụng Của Cây Nhị Phân.....	28
4.6.2. Ứng Dụng Của Cây Bao Trùm Tối Thiểu.....	29
5. MẬT MÃ VÀ ỨNG DỤNG.....	29
5.1. Nền tảng của mật mã.....	29
5.1.1. Số nguyên tố.....	29
5.1.2. Số học mô đun (Modulo Arithmetic).....	29
5.2. Mật mã học.....	30
5.2.1. Mật mã khóa bí mật (symmetric cryptography).....	30

5.2.2. Mật mã khóa công khai (asymmetric cryptography).....	30
5.3. Hàm băm (hash function).....	31
5.4. Chữ ký số (digital signature).....	31
5.5. Ứng dụng thực tế.....	32
PHẦN 2 - CHỦ ĐỀ NGHIÊN CỨU SÂU: NHỮNG TÌM HIỂU NGHIÊN CỨU KỸ VÀ SÂU VỀ 1 CHỦ ĐỀ, VẤN ĐỀ, BÀI TOÁN. CÓ THỂ CHỌN 1 TRONG HAI HƯỚNG SAU:.....	33
I. Thuật toán tham lam + heuristic.....	33
II. Thuật toán DSATUR (Degree of Saturation).....	35
III. Ứng dụng vào bài toán.....	37
1. Dữ liệu đầu vào.....	37
2. Lựa chọn thuật toán.....	37
3. Các ràng buộc.....	38
4. Các bước thực hiện.....	38
4.1. Tiền xử lý dữ liệu.....	38
4.2. Xây dựng đồ thị xung đột.....	38
4.3. Sắp xếp lịch thi và tối ưu hóa ca thi.....	39
4.4. Phân bổ phòng thi theo quy mô và sức chứa.....	39
4.5. Đánh giá độ phức tạp.....	40
4.6. Kết quả.....	42
4.7. Thống kê và nhận xét.....	42
4.8. Kết luận.....	48
Tài liệu tham khảo.....	49

PHẦN 1 - TỔNG QUAN MÔN HỌC: TÓM LƯỢC CÁC NỘI DUNG ĐÃ LĨNH HỘI ĐƯỢC TỪ MÔN HỌC (VỀ LÝ THUYẾT, THUẬT TOÁN VÀ ỨNG DỤNG)

1. PHÂN TÍCH THUẬT TOÁN

1.1. Khái niệm và vai trò

Thuật toán là một dãy hữu hạn các bước xác định nhằm giải một bài toán cụ thể. Một thuật toán tốt không chỉ đưa ra kết quả đúng mà còn phải thực hiện trong thời gian và tài nguyên hợp lý. Một thuật toán cần đảm bảo:

- Có input và output rõ ràng
- Tính hữu hạn
- Tính xác định
- Có thể cài đặt được

1.2. Mục tiêu của phân tích thuật toán

Khi kích thước dữ liệu nhỏ, hầu như mọi thuật toán đều có thể chạy được. Tuy nhiên khi dữ liệu tăng (đến hàng triệu, hàng tỷ bản ghi), sự khác biệt giữa các thuật toán trở nên cực kỳ quan trọng. Việc phân tích thuật toán giúp đánh giá và dự đoán được kết quả của chương trình trước khi triển khai thực tế.

Phân tích thuật toán nhằm trả lời hai câu hỏi cơ bản:

- Thuật toán có đúng không? → Chứng minh tính đúng
- Thuật toán có hiệu quả không? → Phân tích độ phức tạp (thời gian/không gian)

1.3. Chứng minh tính đúng của thuật toán

Một thuật toán chỉ có giá trị khi nó luôn cho kết quả đúng với mọi đầu vào hợp lệ. Kiểm thử chỉ kiểm tra được một số trường hợp cụ thể, trong khi chứng minh toán học đảm bảo tính tổng quát.

Cụ thể:

	Kiểm thử	Chứng minh toán học
Cách thực hiện	Chạy thử thuật toán với các dữ liệu vào cụ thể	chứng minh bằng toán học
Ưu điểm	Dễ thực hiện	Tổng quát
Nhược điểm	Có thể không phát hiện hết tất cả lỗi	Khó hơn, có thể có lỗi

Do đó, nên kết hợp kiểm thử và chứng minh tính đúng toán học để tăng hiệu

quả.

1.3.1. Thuật toán đệ quy dùng quy nạp toán học

Đệ quy thường dựa trên việc chia bài toán thành các bài toán con nhỏ hơn. Do đó, phương pháp quy nạp toán học là công cụ tự nhiên để chứng minh.

Cấu trúc:

- Cơ sở quy nạp: đúng với trường hợp nhỏ nhất.
- Giả thiết quy nạp: đúng với kích thước n .
- Bước tổng quát: chứng minh đúng với $n+1$.

Ta chứng minh thuật toán đúng với kích thước nhỏ nhất trước, sau đó dựa trên giả thiết rằng nó đúng với n để suy ra đúng với $n+1$. Nếu cả hai bước đều đúng, thuật toán đúng với mọi n .

1.3.2. Thuật toán không đệ quy dùng bất biến vòng lặp

Bất biến vòng lặp là biểu thức logic có giá trị không đổi trong quá trình lặp, luôn đúng trước và sau mỗi lần lặp.

Cách thực hiện gồm 3 bước:

- Khởi tạo: đúng trước khi vòng lặp bắt đầu.
- Duy trì: nếu đúng ở lần lặp này thì vẫn đúng ở lần sau.
- Kết thúc: khi vòng lặp dừng, bất biến giúp ta suy ra kết quả.

Bất biến vòng lặp giống như “trục xương sống” của vòng lặp. Nếu ta chứng minh được bất biến luôn đúng, ta có thể đảm bảo thuật toán hoạt động chính xác.

1.4. Phân tích độ phức tạp của thuật toán

Độ phức tạp thời gian là số thao tác cơ bản thực hiện theo kích thước dữ liệu n , đo bằng tốc độ tăng trưởng khi n tăng.

Có ba trường hợp:

- Tốt nhất (best case)
- Trung bình (average case)
- Xấu nhất (worst case)

Trong thực tế, ta thường quan tâm đến trường hợp xấu nhất để đảm bảo hệ thống không vượt quá giới hạn tài nguyên.

Có hai phương pháp đánh giá thời gian thực hiện thuật toán:

	Thực nghiệm	Lý thuyết
Cách thực hiện	Lập trình, chạy thử nghiệm, ghi nhận các số đo về độ phức tạp	Xác định quan hệ về thời gian với kích thước dữ liệu bằng hàm toán học
Ưu điểm	Có thể đánh giá được tất cả các thuật toán khả trình	không phụ thuộc ngôn ngữ lập trình, loại máy tính; đánh giá được với dữ liệu có kích thước lớn

Nhược điểm	Bị hạn chế và phụ thuộc vào máy tính thực hiện	Khó vì biểu diễn các ràng buộc về mặt toán học
------------	--	--

Khi n rất lớn, chỉ quan tâm đến phần tăng nhanh nhất của hàm để đánh giá thời gian thực hiện, tài nguyên cần thiết để □ phương pháp lý thuyết sẽ sử dụng ký pháp tiệm cận để xác định mối quan hệ giữa các hàm đánh giá độ phức tạp. Các ký pháp biểu diễn tiệm cận chính:

	Big-O – Cận trên	Omega (Ω) – Cận dưới	Theta (Θ) – Cận chặt
Ý nghĩa	Biểu diễn tốc độ tăng tối đa → Cho biết thuật toán không chạy chậm hơn một mức nào đó.	Biểu diễn tốc độ tăng tối thiểu.	Khi hàm có cùng tốc độ tăng trưởng ở cả cận trên và dưới.

Quy tắc tính độ phức tạp

- Bỏ hằng số ($2n \approx n$).
- Giữ bậc cao nhất ($n^2 + n \approx n^2$).
- Vòng lặp lồng nhau → nhân độ phức tạp.

1.5. Phân loại bài toán P và NP (độ phức tạp P, NP)

	Bài toán P	Bài toán lớp NP
Định nghĩa	Bài toán quyết định giải được trong thời gian đa thức ($O(n^k)$). (có tính dễ)	Bài toán quyết định khó tìm lời giải nhưng dễ kiểm tra lời giải. Hay, là bài toán quyết định giải bằng thuật toán không đơn định với thời gian đa thức
Ví dụ		Tìm các nhân tử nguyên tố của 13717421 – Phức tạp Nhưng dễ kiểm tra rằng $3607 \times 3803 = 13717421$
Khi $N=NP$	Nhiều bài toán tối ưu sẽ giải được nhanh Nhưng hệ mật mã hiện nay có thể bị phá vỡ. → Một trong những bài toán lớn nhất của khoa học máy tính	

Tóm lại, Mục tiêu của thuật toán là tìm một quy trình hữu hạn để giải bài toán một cách đúng đắn và hiệu quả. Một thuật toán tốt cần đảm bảo:

- Tính đúng (Correctness)
- Tính dừng (Termination)
- Hiệu quả về thời gian và bộ nhớ (Efficiency)

Quá trình giải một bài toán thường gồm hai bước:

- Thiết kế thuật toán – tìm cách giải
- Phân tích thuật toán – chứng minh đúng và đánh giá hiệu năng

Hai bước này có mối quan hệ chặt chẽ:

- Thiết kế tốt, nhưng phân tích kém $\rightarrow$ không biết có khả thi hay không?
- Phân tích tốt nhưng thiết kế sai $\rightarrow$ thuật toán vô nghĩa

2. MỘT SỐ PHƯƠNG PHÁP THIẾT KẾ THUẬT TOÁN CƠ BẢN

Các phương pháp thiết kế được coi là chiến lược tư duy để xây dựng thuật toán quyết định bài toán có tìm được lời giải không.

2.1. Các phương pháp chính xác

Là các phương pháp thuộc nhóm thuật toán chính xác, tức là luôn tìm ra nghiệm đúng hoặc nghiệm tối ưu của bài toán (nếu tồn tại)

Các phương pháp khác nhau ở cách tiếp cận không gian bài toán và chiến lược tìm lời giải.

2.1.1. Kỹ thuật đệ quy

Đệ quy là kỹ thuật giải bài toán bằng cách gọi lại chính nó với bài toán nhỏ hơn.

Điều kiện áp dụng:

- Bài toán có cấu trúc tự đồng dạng
- Có thể thu nhỏ kích thước
- Có trường hợp cơ sở

Tư duy cốt lõi

Nếu bài toán $P(n)$ có thể biểu diễn qua $P(n-1)$ hoặc $P(n/k)$, ta có thể dùng đệ quy.

Công thức độ phức tạp thường ở dạng:

$$T(n) = T(n-1) + c, \text{ hoặc } T(n) = aT(n/b) + g(n)$$

Vai trò: Đệ quy là nền tảng của nhiều phương pháp khác:

- Chia để trị = đệ quy nhiều nhánh
- Quy hoạch động = đệ quy + ghi nhớ
- Quay lui = đệ quy duyệt không gian nghiệm

2.1.2. Phương pháp chia để trị và quy hoạch động (phương pháp phân rã bài toán)

Tư duy chung: Phân rã bài toán thành các bài toán con để giải

	Chia để trị	Quy hoạch động
Ý tưởng cơ bản	- Chia bài toán thành các bài toán con độc lập - Giải từng bài toán con (thường bằng đệ quy) - Kết hợp kết quả (không quan tâm đến việc có lặp lại không) → Tư duy phân rã cấu trúc bài toán (tập trung chia nhỏ)	- Xác định các bài toán con (có chồng lặp) - Nếu bài toán con bị lặp lại → chỉ giải một lần - Lưu kết quả và tái sử dụng → Tư duy tối ưu hóa và tái sử dụng kết quả (tập trung tránh lặp)
Mục tiêu	Giảm kích thước bài toán	Tối ưu hóa tính toán
Đặc điểm	- Bài toán con thường độc lập - Không lưu kết quả - Triển khai: top-down - Bộ nhớ ít	- Bài toán con có chồng lặp - Lưu kết quả - Triển khai: top-down hoặc bottom-up - Bộ nhớ nhiều
Ví dụ	- Merge Sort → $O(n \log n)$ - Quick Sort (trung bình $O(n \log n)$) - Binary Search → $O(\log n)$	- Fibonacci tối ưu → $O(n)$ - Knapsack 0-1
Ưu điểm	- Thiết kế tự nhiên, rõ ràng - Phù hợp bài toán phân rã cấu trúc rõ ràng - Hiệu quả cao nếu bài toán con độc lập - Tốn ít bộ nhớ hơn quy hoạch động	- Tránh lặp tính toán - Giảm độ phức tạp từ hàm mũ xuống đa thức - Giải được nhiều bài toán tối ưu - Hiệu quả cao khi có chồng lặp
Hạn chế	- Có thể lặp lại cùng một bài toán con - Không tối ưu với bài toán có chồng lặp lớn - Không giải quyết trực tiếp bài toán tối ưu phức tạp	- Tốn bộ nhớ để lưu bảng phương án - Khó xác định trạng thái và công thức truy hồi - Không áp dụng được nếu không có cấu trúc con tối ưu

Tóm lại, chia để trị và quy hoạch động đều dựa trên việc phân rã bài toán thành các bài toán con. Tuy nhiên, trong khi chia để trị giải các bài toán con một cách độc lập và có thể lặp lại tính toán, thì quy hoạch động tối ưu hóa quá trình này bằng cách lưu trữ và tái sử dụng kết quả của các bài toán con chồng lặp. Vì vậy, có thể xem quy hoạch động là “chia để trị có ghi nhớ”. Do đó, quy hoạch động thường hiệu quả hơn trong các bài toán tối ưu có cấu trúc con lặp lại, nhưng đánh đổi bằng việc sử dụng nhiều bộ nhớ hơn.

2.1.3. Phương pháp Quay lui và Nhánh cận (nhóm tìm kiếm không gian nghiệm)

Khác so với nhóm chia để trị - quy hoạch động, hai phương pháp này không phân rã bài toán theo cấu trúc toàn học, mà duyệt không gian nghiệm để tìm lời giải. Cụ thể:

- Xây dựng nghiệm từng bước
- Duyệt các khả năng
- Cắt bỏ những nhánh không cần thiết

	Quay lui (Backtracking)	Nhánh cận (Branch and Bound)
Bản chất	- Xây dựng nghiệm từng bước: $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n$ - Kiểm tra tính hợp lệ tại mỗi bước - Nếu một bước không hợp lệ $\rightarrow$ quay lại bước trước.	Nhánh cận là mở rộng của quay lui dành cho bài toán tối ưu. Ngoài việc kiểm tra hợp lệ, phương pháp này còn: - Tính toán một cận (bound) cho mỗi nhánh - Cắt bỏ nhánh không thể tối ưu
Cách thực hiện	Giả sử nghiệm có dạng: $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n$ Tại bước i: - Thử mọi giá trị có thể của x_i - Nếu hợp lệ $\rightarrow$ tiếp tục - Nếu không $\rightarrow$ quay lại thử giá trị khác	Tại mỗi trạng thái: - Tính giá trị tạm thời - Ước lượng giá trị tốt nhất có thể đạt được từ nhánh đó - So sánh với nghiệm tốt nhất hiện có - Nếu không thể tốt hơn $\rightarrow$ cắt nhánh
Mục tiêu	Tìm nghiệm hợp lệ	Tìm nghiệm tối ưu (Không cần duyệt toàn bộ không gian)

Cắt nhánh	Dựa vào ràng buộc hợp lệ có thể vét cạn	Dựa vào cận chi phí (đánh giá tối ưu) hiệu quả thường tốt hơn
Độ phức tạp	Nếu mỗi bước có d lựa chọn: $O(d^n)$ → Thường là hàm mũ.	Phụ thuộc chiến lược cận: - Cách xây dựng hàm cận - Thứ tự duyệt nhánh
Ưu điểm	- Tổng quát, dễ cài đặt - Phù hợp bài toán ràng buộc - Có thể tìm tất cả nghiệm	- Tìm nghiệm tối ưu - Cắt giảm không gian tìm kiếm đáng kể - Thực tế hiệu quả hơn vét cạn
Hạn chế	- Thường có độ phức tạp hàm mũ - Không hiệu quả khi không gian nghiệm lớn - Không đảm bảo tối ưu nếu không duyệt hết	- Vẫn có thể rất chậm trong trường hợp xấu nhất - Khó thiết kế hàm cận tốt - Phụ thuộc vào chiến lược duyệt

Tóm lại, quay lui và nhánh cận là hai phương pháp tìm kiếm nghiệm trong không gian tổ hợp. Quay lui tập trung vào việc tìm nghiệm hợp lệ bằng cách loại bỏ các nhánh không thỏa ràng buộc, trong khi nhánh cận mở rộng phương pháp này bằng cách sử dụng hàm đánh giá để loại bỏ các nhánh không thể dẫn đến nghiệm tối ưu.

Tổng kết, các phương pháp thiết kế thuật toán chính xác có thể chia thành:

- Nhóm dựa trên cấu trúc toán học (Chia để trị, quy hoạch động)
- Nhóm dựa trên tìm kiếm không gian nghiệm (Quay lui, Nhánh cận)

Sự khác biệt lớn nhất giữa các nhóm nằm ở cách tiếp cận bài toán, khả năng tối ưu hóa cũng như mức độ hiệu quả khi dữ liệu lớn.

Một bài toán có thể được giải bằng nhiều phương pháp khác nhau, nhưng chọn đúng phương pháp quyết định hiệu quả thực tế của thuật toán. Do đó, cần dựa vào cấu trúc, mục tiêu bài toán (tối ưu hay liệt kê) và hạn chế về thời gian và bộ nhớ để quyết định lựa chọn nhóm phương pháp, phương pháp thiết kế phù hợp.

2.2. Các phương pháp có sự thỏa hiệp

Các phương pháp kể trên (như chia để trị, quy hoạch động, quy lui, nhánh cận) là các thuật toán chính xác (Exact Algorithms), đảm bảo tìm nghiệm đúng hoặc tìm nghiệm tối ưu.

Tuy nhiên, không phải bài toán nào cũng có thể giải hiệu quả bằng thuật toán chính xác trong thời gian đa thức, đặc biệt với các bài toán lớp NP. Vì vậy,

nhóm phương pháp thoả hiệp ra đời, chấp nhận đánh đổi giữa tính tối ưu, thời gian và độ chính xác

Phương pháp thoả hiệp có thể chia thành 4 hướng lớn, mỗi phương pháp thể hiện một kiểu đánh đổi khác nhau, gồm:

- Tham lam (Greedy) - thoả hiệp chiến lược cục bộ
- Trực tuyến (Online) - thoả hiệp vì thiếu thông tin
- Ngẫu nhiên (Randomized) - thoả hiệp xác suất
- Xấp xỉ (Approximation) - thoả hiệp độ chính xác

	Tham lam (Greedy)	Trực tuyến (Online)	Ngẫu nhiên (Randomized)	Xấp xỉ (Approximation)
Bản chất	- Xây dựng lời giải bằng cách chọn tối ưu cục bộ tại mỗi bước, không quay lại sửa đổi. - Không duyệt toàn bộ không gian nghiệm như quay lui.	- Ra quyết định khi chưa biết toàn bộ dữ liệu tương lai. - Khác với thuật toán offline (biết toàn bộ input từ đầu).	Đưa yếu tố ngẫu nhiên vào quá trình lựa chọn. Hai dạng chính: - Monte Carlo → có thể sai với xác suất nhỏ - Las Vegas → luôn đúng nhưng thời gian ngẫu nhiên	Tìm lời giải gần tối ưu trong thời gian đa thức. Áp dụng cho NP-hard.
Dạng thoả hiệp	- Đánh đổi giữa tốc độ và đảm bảo tối ưu toàn cục - Không xét toàn bộ khả năng	- Đánh đổi giữa thông tin và chất lượng lời giải - Không thể tối ưu toàn cục vì thiếu thông tin - Đánh giá bằng: Competitive ratio (tỉ lệ cạnh tranh)	- Đánh đổi giữa tính xác định và hiệu quả trung bình - Chấp nhận xác suất sai nhỏ hoặc thời gian biến động	- Đánh đổi giữa độ chính xác và thời gian chạy - Đánh giá bằng: Approximation ratio; Sai số tương đối

Khi nào hiệu quả	Khi bài toán có: - Tính chất lựa chọn tham lam - Cấu trúc con tối ưu	- Hệ thống thời gian thực - Dữ liệu đến liên tục - Không thể lưu trữ toàn bộ input	- Khi thuật toán xác định quá chậm - Khi cấu trúc dữ liệu phức tạp - Khi muốn tối ưu thời gian trung bình -	- Bài toán NP-hard - Dữ liệu lớn - Khi “gần đúng là đủ”
Hạn chế	- Không phải lúc nào cũng tối ưu - Phụ thuộc mạnh vào cấu trúc bài toán - Ví dụ đổi tiền (mệnh giá 10,6,1) → tham lam không tối ưu.	- Không đảm bảo tối ưu tuyệt đối - Phụ thuộc vào mô hình đánh giá cạnh tranh	- Không kiểm soát tuyệt đối kết quả - Phụ thuộc vào phân bố xác suất	- Không đạt tối ưu tuyệt đối - Cần chứng minh hệ số xấp xỉ
Ứng dụng	- Lập lịch - Thiết kế mạng - Tối ưu chi phí đơn giản	- Bài toán "Thuê hay mua" - Bài toán "Thư ký"	- Kiểm tra số nguyên tố (Miller–Rabin) - QuickSort ngẫu nhiên - Tìm cặp điểm gần nhất - Mật mã học	- TSP xấp xỉ - Knapsack xấp xỉ - Vertex Cover - Logistics, vận tải

Tóm lại, các phương pháp thỏa hiệp là cách tiếp cận thực tế khi bài toán vượt quá giới hạn tính toán. Chúng cho phép cân bằng giữa: tính tối ưu, thời gian thực hiện, tài nguyên hệ thống, mức độ chính xác chấp nhận được. Do đó, nếu

khả thi thì lựa chọn phương pháp chính xác, nếu không khả thi, chọn dạng thoả hiệp phù hợp.

3. LÝ THUYẾT ĐỒ THỊ VÀ ỨNG DỤNG

Nếu phần phương pháp thiết kế thuật toán cung cấp *công cụ tư duy*, thì đồ thị cung cấp *không gian áp dụng* cho các công cụ đó. Nhiều bài toán thực tế như tìm đường đi, tối ưu chi phí, phân phối tài nguyên hay thiết kế mạng đều có thể được mô hình hóa bằng đồ thị.

3.1. Nhận thức và hiểu biết về đồ thị

3.1.1. Khái niệm cơ bản về đồ thị

Định nghĩa đồ thị: Đồ thị (graph) là một mô hình toán học dùng để biểu diễn các đối tượng và mối quan hệ giữa chúng. Một đồ thị được ký hiệu bởi: $G = (V, E)$

Trong đó:

- V (Vertices/Nodes) là tập các đỉnh, đại diện cho các đối tượng hoặc thực thể.
- E (Edges) là tập các cạnh, biểu diễn mối quan hệ giữa các đỉnh.

Các khái niệm cơ bản

- Đỉnh (Vertex): là đơn vị cơ bản của đồ thị, đại diện cho một đối tượng hoặc thực thể. Ví dụ: thành phố, máy tính, người dùng trong mạng xã hội.
- Cạnh (Edge): là mối liên kết giữa hai đỉnh, biểu diễn quan hệ hoặc sự tương tác giữa các đối tượng. Cạnh được chia thành hai loại:
 - Cạnh có hướng (Directed edge): biểu diễn quan hệ một chiều.
 - Cạnh không hướng (Undirected edge): biểu diễn quan hệ hai chiều.
- Trọng số (Weight): là giá trị gán cho cạnh, dùng để biểu diễn chi phí, khoảng cách, thời gian hoặc mức độ ưu tiên của mối liên kết giữa hai đỉnh.
- Bậc của đỉnh (Degree): là số cạnh liên kết trực tiếp với một đỉnh.
 - Trong đồ thị vô hướng: bậc của đỉnh là số cạnh kề với đỉnh đó.
 - Trong đồ thị có hướng:
 - In-degree: số cạnh đi vào đỉnh.
 - Out-degree: số cạnh đi ra từ đỉnh.
- Đường đi (Path): là một dãy các đỉnh liên tiếp được nối với nhau bởi các cạnh, thể hiện khả năng di chuyển hoặc kết nối giữa hai đỉnh.
- Chu trình (Cycle): là một đường đi trong đó đỉnh bắt đầu và đỉnh kết thúc trùng nhau, cho thấy sự tồn tại của vòng lặp trong đồ thị.
- Đồ thị vô hướng liên thông: là đồ thị mà giữa mọi cặp đỉnh bất kỳ đều tồn tại ít nhất một đường đi.

- Cây (Tree): là đồ thị vô hướng liên thông và không chứa chu trình, thường được dùng để biểu diễn cấu trúc phân cấp như cây thư mục hoặc sơ đồ tổ chức.

3.1.2. Cấu trúc dữ liệu biểu diễn đồ thị

Có ba cấu trúc dữ liệu phổ biến để biểu diễn đồ thị:

- Ma trận kề (Adjacency Matrix)

Đồ thị được biểu diễn bằng ma trận vuông kích thước $n \times n$, trong đó phần tử $A[i][j]$ cho biết sự tồn tại (hoặc trọng số) của cạnh giữa đỉnh i và j .

- Danh sách kề (Adjacency List)

Mỗi đỉnh được gắn với một danh sách các đỉnh kề (kèm trọng số nếu có). Cấu trúc này chỉ lưu các cạnh thực sự tồn tại nên tiết kiệm bộ nhớ với đồ thị thưa.

- Danh sách cạnh (Edge List)

Đồ thị được biểu diễn bằng danh sách các cạnh dưới dạng (u, v, w) , trong đó u và v là hai đỉnh, w là trọng số của cạnh.

Tiêu chí	Ma trận kề	Danh sách kề	Danh sách cạnh
Không gian	$O(V^2)$	$O(V+E)$	$O(E)$
Kiểm tra cạnh	$O(1)$	$O(\deg(u))$	$O(E)$
Duyệt đỉnh kề	$O(V)$	$O(\deg(u))$	$O(E)$
Phù hợp	Đồ thị dày	Đồ thị dày	Xử lý cạnh

3.1.3. Mô hình và ứng dụng của đồ thị trong thực tiễn

Trong các bài toán thực tế, đồ thị được sử dụng để mô hình hóa các đối tượng và mối quan hệ giữa chúng, từ đó cho phép áp dụng các thuật toán tối ưu và phân tích mạng.

Mạng máy tính và viễn thông

- Đỉnh: máy tính, router, server.
- Cạnh: đường truyền.
- Trọng số: độ trễ, băng thông, chi phí truyền.

Các thuật toán như Dijkstra hoặc Bellman–Ford được dùng để tìm đường truyền tối ưu.

Giao thông và bản đồ số

Đồ thị được dùng để biểu diễn bản đồ, trong đó các thuật toán Dijkstra hoặc A* giúp tìm lộ trình ngắn nhất và thích ứng với tình trạng giao thông.

Mạng xã hội

- Đỉnh: người dùng.
- Cạnh: quan hệ bạn bè hoặc theo dõi.

Các thuật toán BFS, DFS, PageRank hoặc phát hiện cộng đồng được dùng để phân tích mức độ ảnh hưởng và kết nối.

Quản lý dự án và lập lịch công việc

- Đồ thị có hướng không chu trình (DAG) được dùng để biểu diễn quan hệ phụ thuộc giữa các công việc, áp dụng các thuật toán Topological Sort và Critical Path Method

Trí tuệ nhân tạo và khoa học máy tính

Đồ thị được dùng để biểu diễn không gian trạng thái, cây tìm kiếm và đồ thị tri thức; các thuật toán DFS, BFS, A* và Minimax giúp tìm lời giải tối ưu cho bài toán.

Từ khái niệm cơ bản trên, một số bài toán quan trọng trên đồ thị được đặt ra:

- **Đường đi trên đồ thị:** Xác định cách di chuyển từ một đỉnh đến đỉnh khác thông qua các cạnh.
- **Đường đi ngắn nhất:** Tìm con đường có tổng chi phí nhỏ nhất giữa hai đỉnh, thường được giải bằng các thuật toán tham lam như Dijkstra hoặc quy hoạch động như Floyd–Warshall.
- **Chu trình trong đồ thị:** Xác định sự tồn tại của chu trình Hamilton (đi qua mọi đỉnh đúng một lần) hoặc chu trình Euler (đi qua mọi cạnh đúng một lần), thể hiện các mức độ phức tạp khác nhau của bài toán.

3.2. Đường đi trong đồ thị

3.2.1. Khái niệm

Cho đồ thị $G=(V,E)$

Đường đi (Path) là dãy đỉnh: $v_1, v_2, \dots, v_k$ sao cho mỗi cặp đỉnh liên tiếp có cạnh nối.

Độ dài đường đi:

- Đồ thị không trọng số $\rightarrow$ số cạnh
- Đồ thị trọng số $\rightarrow$ tổng trọng số

Đường đi đơn (simple path): không lặp đỉnh

Mục tiêu bài toán

- Kiểm tra tồn tại đường đi giữa hai đỉnh?
- Tìm tất cả đường đi?
- Tìm đường đi ngắn nhất?

3.2.2. Các thuật toán cơ bản

- DFS (Depth-First Search)
- BFS (Breadth-First Search)

	DFS (Depth-First Search) (duyệt theo chiều sâu)	BFS (Breadth-First Search) (duyệt theo chiều rộng)
Ý tưởng	- Đi sâu tối đa theo một nhánh trước - Khi không đi tiếp được thì quay lui (backtracking)	- Duyệt theo từng tầng - Xét tất cả đỉnh cách nguồn 1 cạnh - Sau đó cách 2 cạnh - Sau đó cách 3 cạnh...
Độ phức tạp	$O(V+E)$ Vì mỗi đỉnh và cạnh chỉ được duyệt tối đa một lần.	$O(V+E)$
Đặc điểm	- Tìm được một đường đi, nhưng không đảm bảo ngắn nhất - Phù hợp kiểm tra liên thông - Dễ phát hiện chu trình - Phù hợp với cấu trúc đệ quy	- Tìm được đường đi ngắn nhất trong đồ thị không trọng số - Xây dựng cây BFS theo lớp khoảng cách
Khi nào sử dụng	- Kiểm tra đồ thị có liên thông? - Tìm thành phần liên thông - Phát hiện chu trình - Bài toán quay lui (Hamilton, mê cung...) ⇒ phù hợp cho phân tích cấu trúc và bài toán đệ quy	- Tìm số bước ít nhất - Tìm khoảng cách tối thiểu - Bài toán lan truyền (virus, tin tức...) ⇒ phù hợp khi cần tìm đường đi ngắn nhất trong đồ thị không trọng số
Cấu trúc dữ liệu	Stack / Đệ quy	- Queue
Ưu điểm	- Tiết kiệm bộ nhớ - Phù hợp bài toán cấu trúc - Cài đặt đơn giản	- Tìm đường đi ngắn nhất trong đồ thị không trọng số - Phù hợp bài toán khoảng cách tối thiểu - Dễ xác định level của đỉnh
Hạn chế	- Không đảm bảo đường đi ngắn nhất - Có thể rơi vào nhánh rất sâu	- Tốn bộ nhớ hơn - Không hiệu quả nếu chỉ cần kiểm tra tồn tại đường đi

	- Độ quy sâu có thể gây tràn stack	- Không phù hợp đồ thị rất lớn và phân nhánh mạnh
--	------------------------------------	---

3.2.3. Các bước thực hiện

Bước 1: Xác định loại đồ thị

- Có hướng hay vô hướng?
- Có trọng số hay không?

Nếu không trọng số, ta dùng DFS hoặc BFS.

Bước 2: Chọn thuật toán

Trường hợp 1: Chỉ cần kiểm tra tồn tại đường đi

→ Dùng DFS hoặc BFS

Vì cả hai có độ phức tạp: $O(V+E)$

Nếu trong quá trình duyệt gặp đỉnh $t \rightarrow$ tồn tại đường đi.

Trường hợp 2: Cần tìm đường đi ngắn nhất (không trọng số)

→ Dùng BFS

Vì BFS duyệt theo từng lớp khoảng cách tăng dần nên đảm bảo: số cạnh ít nhất

Bước 3: Truy vết đường đi

Khi duyệt (DFS/BFS), lưu mảng $parent[]$:

$parent[v] = u$ nghĩa là đến v từ u

Sau khi tìm được t , truy ngược từ t về s

3.2.4. Ứng dụng trong thực tế

Kiểm tra liên thông

- Mạng máy tính: hai máy có kết nối được không?
- Hệ thống giao thông: có tuyến đường giữa hai địa điểm?
- Phân tích mạng xã hội: hai người có quan hệ gián tiếp?

Phát hiện chu trình

- Kiểm tra deadlock trong hệ điều hành
- Kiểm tra phụ thuộc vòng trong biên dịch
- Phát hiện vòng lặp trong mạng

Thành phần liên thông

- Phân cụm người dùng trong mạng xã hội
- Xác định nhóm hệ thống độc lập

3.3. Đường đi ngắn nhất trong đồ thị

Trong nhiều bài toán thực tế, ta cần tìm một đường đi giữa hai đỉnh sao cho tổng chi phí là nhỏ nhất. Bài toán này được gọi là bài toán tìm đường đi ngắn nhất (Shortest Path Problem).

3.3.1. Phát biểu bài toán

Cho một đồ thị có trọng số $G = (V, E, w)$, trong đó:

- V là tập các đỉnh,
- E là tập các cạnh,
- $w : E \rightarrow \mathbb{R}^+$ là hàm gán trọng số (chi phí, khoảng cách, thời gian) cho mỗi cạnh.

Với hai đỉnh $s, t \in V$, hãy tìm một đường đi từ s đến t sao cho tổng trọng số các cạnh trên đường đi là nhỏ nhất.

3.3.2. Các thuật toán chính

Tiêu chí	Dijkstra	Bellman–Ford	Floyd–Warshall
Định nghĩa	Là một thuật toán tham lam (greedy) dùng để tìm đường đi ngắn nhất từ một đỉnh nguồn đến các đỉnh còn lại trong đồ thị có trọng số không âm	Là thuật toán cho phép tìm đường đi ngắn nhất trong đồ thị có trọng số âm, miễn là không tồn tại chu trình âm.	Là thuật toán dùng để tìm đường đi ngắn nhất giữa mọi cặp đỉnh trong đồ thị
Ý tưởng	- Luôn chọn đỉnh chưa xét có nhãn khoảng cách nhỏ nhất. - Cố định khoảng cách ngắn nhất đến đỉnh đó. - Cập nhật khoảng cách đến các đỉnh kề	Thư giãn cạnh lặp lại $ V - 1$ lần	- Quy hoạch động - Xét từng đỉnh trung gian Công thức: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$
Độ phức tạp	- Dùng mảng: $O(V^2)$ - Dùng heap: $O((V+E)\log V)$	$O(VE)$	$O(V^3)$

Ưu điểm	- Nhanh - Hiệu quả trong thực tế (GPS, định tuyến mạng)	- Xử lý được trọng số âm - Phát hiện chu trình âm	Tìm đường đi ngắn nhất giữa mọi cặp đỉnh
Hạn chế	Không xử lý được cạnh âm	Chậm hơn Dijkstra	- Tốn bộ nhớ - Không phù hợp đồ thị lớn

3.3.3. Cách giải bài toán

Bước 1: Xác định loại đồ thị

- Có trọng số?
- Có trọng số âm?
- Cần 1 nguồn hay mọi cặp?

Bước 2: Chọn thuật toán phù hợp

- Không trọng số → BFS
- Trọng số dương → Dijkstra
- Có âm → Bellman–Ford
- Mọi cặp → Floyd

Bước 3: Phân tích độ phức tạp

So sánh với kích thước đồ thị để đánh giá tính khả thi.

3.3.4. Ứng dụng trong thực tế

Hệ thống định vị (GPS)

- Google Maps, Grab, Be...
- Tìm tuyến đường nhanh nhất hoặc ngắn nhất
- Tối ưu theo:

Khoảng cách

Thời gian

Chi phí

→ Dùng Dijkstra hoặc A*.

Định tuyến mạng Internet

- Router tìm đường truyền dữ liệu ngắn nhất
- Thuật toán OSPF dùng Dijkstra

→ Tối ưu độ trễ và băng thông

Logistics và vận tải

- Tối ưu giao hàng

- Phân phối hàng hóa
- Tối ưu tuyến xe buýt, tàu hỏa

Trí tuệ nhân tạo (AI)

- Game: tìm đường cho nhân vật
- Robot tự hành
- Hệ thống dẫn đường tự động

Mạng xã hội

- Tính “degree of separation”
- Gợi ý bạn bè
- Phân tích mức độ ảnh hưởng

3.4. Chu trình trong đồ thị

3.4.1. Khái niệm

Chu trình (cycle) là đường đi:

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$$

trong đó:

- Các đỉnh trung gian không lặp lại
- Đỉnh đầu = đỉnh cuối

Ý nghĩa:

- Biểu diễn tính tuần hoàn
- Biểu diễn phụ thuộc vòng
- Biểu diễn hành trình khép kín

3.4.2. Phát hiện chu trình

Trong đồ thị vô hướng: Khi duyệt DFS:

- Nếu từ đỉnh u đi sang v
- Và gặp một đỉnh đã thăm
- Nhưng v không phải cha của u

→ Tồn tại chu trình (vì tạo thành vòng khép kín)

Trong đồ thị có hướng: Phải dùng 3 trạng thái:

- Chưa thăm
- Đang thăm (trong stack đệ quy)
- Đã hoàn tất

Nếu gặp lại đỉnh đang “đang thăm” → tồn tại chu trình.

3.4.3. Chu trình Euler

Định nghĩa: Đường Euler trong một đồ thị là một đường đi sao cho mỗi cạnh của đồ thị được đi qua đúng một lần và không lặp lại cạnh nào.

Chu trình Euler là một trường hợp đặc biệt của đường Euler. Là đường đi thỏa mãn điều kiện đi qua tất cả các cạnh đúng một lần, và quay trở lại đúng đỉnh xuất phát ban đầu.

→ chỉ quan tâm đến cạnh, không quan tâm đỉnh

⇒ hai định lý:

Định lý 1 (Đồ thị vô hướng): Một đồ thị vô hướng liên thông $G = (V, E)$ có chu trình Euler khi và chỉ khi mọi đỉnh của đồ thị đều có bậc chẵn, tức là: Mỗi đỉnh trong đồ thị đều có số cạnh kề với nó là một số chẵn.

→ Hệ quả 1 (Đồ thị vô hướng): Một đồ thị vô hướng liên thông $G = (V, E)$ có đường đi Euler khi và chỉ khi nó có đúng hai đỉnh bậc lẻ, còn tất cả các đỉnh còn lại đều có bậc chẵn.

Định lý 2 (Đồ thị có hướng): Một đồ thị có hướng liên thông yếu $G = (V, E)$ có chu trình Euler khi và chỉ khi tại mọi đỉnh, bán bậc ra bằng bán bậc vào. Nói cách khác, với mỗi đỉnh của đồ thị, số cạnh đi ra khỏi đỉnh đó phải bằng số cạnh đi vào đỉnh đó. liên thông yếu thì ta có thể xây dựng được một chu trình Euler.

→ Hệ quả 2 (Đồ thị có hướng): Một đồ thị có hướng liên thông yếu $G = (V, E)$ có **đường đi Euler nhưng không có chu trình Euler** khi và chỉ khi tồn tại đúng hai đỉnh đặc biệt s và t sao cho:

- Tại đỉnh s : số cạnh đi ra nhiều hơn số cạnh đi vào đúng một đơn vị (đỉnh bắt đầu),
- Tại đỉnh t : số cạnh đi vào nhiều hơn số cạnh đi ra đúng một đơn vị (đỉnh kết thúc),

Ứng dụng trong thực tế

- Lập lộ trình giao thông và vận chuyển: tìm lộ trình đi qua tất cả các tuyến đường đúng một lần và quay về điểm xuất phát. Ứng dụng phổ biến trong việc lập đường đi cho xe thu gom rác, xe quét đường, xe phát thư hoặc xe bảo trì hạ tầng giao thông, giúp tiết kiệm thời gian và nhiên liệu.
- Robot và tự động hóa (ví dụ: robot kiểm tra đường ống hoặc robot khảo sát khu vực)
- Thiết kế mạch điện và mạng dây: tối ưu hóa việc đi dây
- Tin sinh học (Bioinformatics): lắp ráp chuỗi ADN từ các đoạn nhỏ thông qua đồ thị De Bruijn → ứng dụng rất quan trọng trong giải mã gen và nghiên cứu sinh học phân tử.
- Bài toán lập lịch và phân công công việc

- Trò chơi và bài toán giải đố: trò chơi nối điểm, vẽ hình bằng một nét hoặc bài toán mê cung

3.4.4. Chu trình Hamilton

Cho đồ thị $G = (V, E)$ có n đỉnh. Ta có:

- Một chu trình $x_1, x_2, \dots, x_n$ được gọi là chu trình Hamilton nếu là đường Hamilton **bắt đầu và kết thúc tại cùng một đỉnh**.
- Đường đi $x_1, x_2, \dots, x_n$ được gọi là đường đi Hamilton đường đi trong đồ thị đi qua **mỗi đỉnh đúng một lần**.

⇒ Khác với Euler, ở đây phải chọn thứ tự đỉnh.

So sánh hai chu trình:

	Euler	Hamilton
Tập trung vào	Cạnh	Đỉnh
Kiểm tra điều kiện	Dựa vào bậc	Không có điều kiện tổng quát
Độ phức tạp	Đa thức	Hàm mũ
Phương pháp	Xây dựng trực tiếp	Quay lui/ Nhánh cận
Lớp bài toán	P	NP-hard
Khi nào sử dụng	Cấu trúc bài toán đơn giản	Bài toán mang bản chất tổng hợp, phải duyệt không gian nghiệm

Ứng dụng trong thực tế:

- Bài toán người du lịch (Travelling Salesman Problem – TSP): trong đó người bán hàng cần đi qua tất cả các thành phố đúng một lần và quay trở về điểm xuất phát sao cho tổng quãng đường là ngắn nhất → ứng dụng rộng rãi trong logistics, giao hàng và lập kế hoạch vận chuyển.
- Lập kế hoạch lộ trình và định tuyến: xây dựng lộ trình đi qua tất cả các điểm cần kiểm tra hoặc giao hàng mà không lặp lại vị trí.
- Thiết kế vi mạch và bo mạch điện tử: sắp xếp vị trí các linh kiện và đường dẫn tín hiệu sao cho tín hiệu đi qua tất cả các điểm cần thiết mà không lặp lại, giúp giảm độ trễ và tiết kiệm không gian.
- Sinh học và hóa học tính toán: mô hình hóa cấu trúc phân tử, phân tích trình tự ADN và giải quyết các bài toán liên quan đến việc sắp xếp các thành phần sinh học theo thứ tự tối ưu.

- Lập lịch và tối ưu hóa tài nguyên: xây dựng thứ tự thực hiện các nhiệm vụ sao cho mỗi công việc chỉ được thực hiện một lần, tránh trùng lặp
- Trí tuệ nhân tạo và robot: lập kế hoạch đường đi qua tất cả các điểm quan trọng trong môi trường mà không quay lại vị trí đã đi qua

4. CÂY VÀ ỨNG DỤNG

4.1. Khái Niệm Cây (Tree)

4.1.1. Định nghĩa

Cây là một đồ thị vô hướng liên thông và không có bất kỳ chu trình nào. Điều này có nghĩa là:

- Không có chu trình: không tồn tại đường đi khép kín qua các đỉnh.
- Chỉ có duy nhất một đường đi giữa hai đỉnh bất kỳ.
- Với n đỉnh, một cây luôn có đúng $n-1$ cạnh.

4.1.2. Cây Có Gốc (Rooted Tree)

Cây có gốc là cây trong đó một đỉnh đặc biệt được chỉ định làm gốc (root), và mọi cạnh đều hướng ra từ gốc đến các đỉnh khác. Các khái niệm liên quan:

- Gốc (Root): Đỉnh đặc biệt không có cha.
- Nút cha (Parent): Đỉnh nằm ngay trên một đỉnh trong cây.
- Nút con (Child): Đỉnh nằm ngay dưới một đỉnh trong cây.
- Lá (Leaf): Đỉnh không có con.
- Nút trong (Internal Node): Đỉnh có ít nhất một con.
- Mức (Level): Khoảng cách từ gốc đến đỉnh đó.
- Chiều cao (Height): Mức lớn nhất trong cây.

4.2. Cây Nhị Phân (Binary Tree)

4.2.1. Định Nghĩa

Cây nhị phân (Binary Tree hay 2-ary Tree) là cây có gốc trong đó:

- Có đúng một nút gốc (root).
- Mỗi nút có tối đa 2 con.
- Hai con của mỗi nút được phân biệt là: con trái (left child) và con phải (right child).

4.2.2. Các Loại Cây Nhị Phân

a) Cây Nhị Phân Đầy Đủ (Full Binary Tree)

Tất cả các nút trong ở mức nhỏ hơn h (chiều cao cây) đều có đủ hai con. Tính chất quan trọng:

- Số nút trên cây đầy đủ chiều cao h : $N = 2^h - 1$ (tổng quát: $N = 2^h - 1$).
- Chiều cao của cây nhị phân đầy đủ N nút là $O(\log N)$.
- Ví dụ: $h = 4$ thì $N = 15$ nút.

b) Cây Nhị Phân Hoàn Chính (Complete Binary Tree)

Cây nhị phân hoàn chỉnh có đặc điểm:

- Các nút được điền đủ từ gốc đến mức $h-1$.
- Mức h (mức cuối) được điền từ trái sang phải, không nhất thiết phải đầy đủ.

4.2.3. Cài Đặt Cây Nhị Phân

a) Cài Đặt Bằng Cấu Trúc Liên Kết

Mỗi nút trong cây được biểu diễn bằng một cấu trúc (struct/object) gồm các trường:

- element: Dữ liệu lưu tại nút.
- parent: Con trỏ đến nút cha.
- left: Con trỏ đến con trái.
- right: Con trỏ đến con phải.

Ưu điểm: Linh hoạt, phù hợp với cây có cấu trúc động. Nhược điểm: Cần quản lý bộ nhớ phức tạp hơn.

b) Cài Đặt Bằng Mảng

Cây nhị phân có thể được lưu trữ trong một mảng. Có hai cách tiếp cận:

- Cách 1 - Mảng với các trường Parent/Left/Right: Mỗi phần tử mảng lưu dữ liệu và chỉ số của cha, con trái, con phải. Giá trị -1 biểu thị null.
- Cách 2 - Mảng đánh số từ 1 (hiệu quả cho cây đầy đủ): Nút gốc ở vị trí 1. Với nút ở vị trí i : Parent ở $i/2$, Left child ở $2i$, Right child ở $2i+1$.

Ưu điểm của mảng: Dễ dàng truy cập các nút theo chỉ số. Nhược điểm: Tốn bộ nhớ khi cây không đầy đủ.

Phương pháp	Ưu điểm	Nhược điểm
Cấu trúc liên kết	Linh hoạt, tiết kiệm bộ nhớ	Quản lý bộ nhớ phức tạp
Mảng (tổng quát)	Dễ truy cập theo chỉ số	Tốn bộ nhớ
Mảng (cây đầy đủ)	Truy cập $O(1)$, hiệu quả cao	Chỉ tốt cho cây đầy đủ

4.3. Thuật toán duyệt cây (Tree Traversal)

Duyệt cây là quá trình thăm tất cả các nút trong cây, mỗi nút đúng một lần. Cây nhị phân có thể được định nghĩa đệ quy: một cây gồm nút gốc, một cây con trái và một cây con phải. Dựa trên cấu trúc này, có 3 phép duyệt chính:

4.3.1. Duyệt Thứ Tự Trước (Pre-order Traversal)

Thứ tự duyệt: Root $\rightarrow$ Left $\rightarrow$ Right (Gốc trước, rồi cây con trái, rồi cây con phải).

Thuật toán:

- Thăm nút gốc.
- Duyệt đệ quy cây con trái.
- Duyệt đệ quy cây con phải.

Mã giả:

```
preorderPrint(bt) { if (bt == null) return; print(bt.element);  
preorderPrint(bt.left); preorderPrint(bt.right); }
```

Ví dụ với cây A(B(D(H,I), E(K)), C(F, G)):

Kết quả Pre-order: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H $\rightarrow$ I $\rightarrow$ E $\rightarrow$ K $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G

4.3.2. Duyệt Thứ Tự Sau (Post-order Traversal)

Thứ tự duyệt: Left $\rightarrow$ Right $\rightarrow$ Root (Cây con trái, cây con phải, rồi gốc cuối cùng).

Thuật toán:

- Duyệt đệ quy cây con trái.
- Duyệt đệ quy cây con phải.
- Thăm nút gốc.

Ví dụ với cùng cây trên:

Kết quả Post-order: H $\rightarrow$ I $\rightarrow$ D $\rightarrow$ K $\rightarrow$ E $\rightarrow$ B $\rightarrow$ F $\rightarrow$ G $\rightarrow$ C $\rightarrow$ A

4.3.3. Duyệt Thứ Tự Giữa (In-order Traversal)

Thứ tự duyệt: Left $\rightarrow$ Root $\rightarrow$ Right (Cây con trái, gốc, rồi cây con phải).

Thuật toán:

- Duyệt đệ quy cây con trái.
- Thăm nút gốc.
- Duyệt đệ quy cây con phải.

Ví dụ với cùng cây trên:

Kết quả In-order: H $\rightarrow$ D $\rightarrow$ I $\rightarrow$ B $\rightarrow$ K $\rightarrow$ E $\rightarrow$ A $\rightarrow$ F $\rightarrow$ C $\rightarrow$ G

4.3.4. So Sánh Ba Phép Duyệt

Phép duyệt	Thứ tự	Ứng dụng tiêu biểu
------------	--------	--------------------

Pre-order	Root $\rightarrow$ Left $\rightarrow$ Right	Sao chép cây, biểu thức tiền tố (prefix)
In-order	Left $\rightarrow$ Root $\rightarrow$ Right	BST cho kết quả sắp xếp tăng dần
Post-order	Left $\rightarrow$ Right $\rightarrow$ Root	Xóa cây, tính giá trị biểu thức (postfix)

4.4. Ứng Dụng Của Cây Nhị Phân

4.4.1. Cây Biểu Thức (*Arithmetic Expression Trees*)

Cây nhị phân có thể biểu diễn các biểu thức toán học. Mỗi nút trong chứa toán tử (+, -, *, /), mỗi nút lá chứa toán hạng (số, biến). Ví dụ biểu thức $((a + b * c) + (d * e + f) * g)$:

- Pre-order duyệt cây $\rightarrow$ ký hiệu tiền tố (prefix/Polish notation).
- In-order duyệt cây $\rightarrow$ ký hiệu trung tố (infix notation) – cách viết thông thường.
- Post-order duyệt cây $\rightarrow$ ký hiệu hậu tố (postfix/Reverse Polish notation), dễ tính bằng ngăn xếp.

4.4.2. Cây Tìm Kiếm Nhị Phân (*Binary Search Tree - BST*)

BST là cây nhị phân với tính chất: với mỗi nút x, tất cả các giá trị trong cây con trái nhỏ hơn x, tất cả giá trị trong cây con phải lớn hơn x. BST cho phép tìm kiếm, thêm, xóa phần tử trong $O(\log N)$ với cây cân bằng.

4.4.3. Sắp Xếp Vun Đống (*Heap Sort*)

Heap là cây nhị phân hoàn chỉnh thỏa mãn tính chất heap: giá trị mỗi nút lớn hơn (max-heap) hoặc nhỏ hơn (min-heap) các con của nó. Heap được sử dụng để sắp xếp với độ phức tạp $O(N \log N)$.

4.5. Cây Bao Trùm Tối Thiểu (*Minimum Spanning Tree - MST*)

4.5.1. Định Nghĩa

Cây bao trùm (Spanning Tree) của đồ thị G là một cây con gồm tất cả các đỉnh của G và một tập con các cạnh của G. Cây bao trùm tối thiểu (MST) của đồ thị liên thông có trọng số G là cây bao trùm có tổng trọng số các cạnh nhỏ nhất trong tất cả các cây bao trùm.

4.5.2. Thuật Toán Prim

Ý tưởng (Chiến lược tham lam - Greedy)

- Bắt đầu với cây T_0 chỉ gồm một đỉnh bất kỳ.

- Lặp lại n-1 lần: tại mỗi bước, tìm cạnh có trọng số nhỏ nhất nối từ cây hiện tại đến một đỉnh chưa thuộc cây, thêm đỉnh và cạnh đó vào cây.
- Dừng khi tất cả n đỉnh đã được thêm vào.

Lược đồ thuật toán

Prim(G): $VT = \{v_0\}; ET = \{\}$

for $i = 1$ to $|V| - 1$:

 Tìm cạnh $e^*(v^*, u^*)$ nhỏ nhất: $v^* \in VT, u^* \notin VT$

$VT = VT \cup \{u^*\}; ET = ET \cup \{e^*\}$

4.5.3. Thuật Toán Kruskal

Ý tưởng

- Sắp xếp tất cả các cạnh theo thứ tự tăng dần của trọng số.
- Bắt đầu với một rừng (forest) rỗng gồm n cây đơn.
- Lần lượt thêm cạnh có trọng số nhỏ nhất vào MST, với điều kiện cạnh đó không tạo thành chu trình.
- Dừng khi đã thêm đủ n-1 cạnh (cây đã liên thông).

Lược đồ thuật toán

Kruskal(G): Sắp xếp E tăng dần; $ET = \{\}$

for $i = 1$ to $n-1$:

 Thêm cạnh e nhỏ nhất không tạo chu trình

4.5.4. So Sánh Prim và Kruskal

Tiêu chí	Prim	Kruskal
Điểm xuất phát	Từ một đỉnh	Từ rừng rỗng
Chiến lược	Mở rộng cây dần	Thêm cạnh nhỏ nhất
Cấu trúc phụ trợ	Hàng đợi ưu tiên	Union-Find (Disjoint Set)
Hiệu quả	Tốt cho đồ thị dày	Tốt cho đồ thị thưa
Độ phức tạp	$O(E \log V)$	$O(E \log E)$

4.6. Ứng Dụng Thực Tiễn

4.6.1. Ứng Dụng Của Cây Nhị Phân

- Tìm kiếm và tra cứu dữ liệu: BST và các biến thể (AVL, Red-Black Tree) được dùng rộng rãi trong cơ sở dữ liệu.
- Nén dữ liệu: Huffman Coding sử dụng cây nhị phân để mã hóa ký tự với độ dài mã thay đổi.

- Đánh giá biểu thức toán học: Compiler và interpreter dùng cây biểu thức để phân tích và tính toán.
- Hàng đợi ưu tiên: Heap (cây nhị phân hoàn chỉnh) được dùng làm hàng đợi ưu tiên trong nhiều thuật toán.

4.6.2. Ứng Dụng Của Cây Bao Trùm Tối Thiểu

- Thiết kế mạng (Network Design): Kết nối các điểm trong mạng máy tính, mạng viễn thông, đường giao thông, lưới điện, hệ thống nước với chi phí tối thiểu.
- Thuật toán xấp xỉ cho bài toán NP-khó: MST được dùng làm nền tảng cho thuật toán xấp xỉ bài toán Người Bán Hàng (TSP).
- Phân cụm dữ liệu (Data Clustering): MST có thể được dùng để phân cụm dữ liệu, bao gồm cả phân tích biểu hiện gen trong tin sinh học.
- Quy hoạch đô thị và hạ tầng: Lập kế hoạch xây dựng mạng lưới đường, điện, nước với chi phí tối ưu.

5. MẬT MÃ VÀ ỨNG DỤNG

Mật mã học hiện đại dựa trên toán rời rạc, đặc biệt là:

- Số nguyên tố
- Số học mô đun
- Bài toán khó tính toán (computational hardness)

5.1. Nền tảng của mật mã

5.1.1. Số nguyên tố

Định nghĩa: Số nguyên tố là số tự nhiên > 1 chỉ chia hết cho 1 và chính nó.

Tính chất quan trọng:

- Mọi số nguyên dương có thể phân tích thành tích các số nguyên tố (Định lý cơ bản của số học).
- Có vô hạn số nguyên tố.
- Việc phân tích một số lớn thành thừa số nguyên tố là rất khó.

Ý nghĩa trong mật mã

Hệ mật mã RSA dựa trên thực tế rằng: Nhân hai số nguyên tố lớn thì dễ, nhưng phân tích tích đó thành thừa số nguyên tố thì khó → Đây là cơ sở bảo mật của mật mã khóa công khai.

5.1.2. Số học mô đun (Modulo Arithmetic)

Khái niệm: Hai số a và b được gọi là đồng dư modulo n nếu:

$$a \equiv b \pmod{n}$$

Tức là: $n \mid (a-b)$

Tính chất

- Có thể cộng, trừ, nhân trong modulo.

- Lũy thừa modulo được tính hiệu quả bằng phương pháp bình phương lặp.

Vai trò trong mật mã

- RSA dùng phép lũy thừa modulo rất lớn.
- Mọi phép mã hóa/giải mã đều diễn ra trong không gian modulo.

5.2. Mật mã học

Mật mã học đảm bảo 4 tính chất:

- Confidentiality – Bí mật
- Integrity – Toàn vẹn
- Authentication – Xác thực
- Non-repudiation – Không chối bỏ

5.2.1. Mật mã khóa bí mật (symmetric cryptography)

Nguyên lý

Dùng cùng một khóa để mã hóa và giải mã.

$$C = E_k(M)$$

$$M = D_k(C)$$

Trong đó:

- k là khóa bí mật
- M là bản rõ
- C là bản mã

Ưu điểm

- Tốc độ nhanh
- Phù hợp mã hóa dữ liệu lớn

Nhược điểm

- Vấn đề phân phối khóa
- Nếu lộ khóa $\rightarrow$ mất an toàn

Ứng dụng

- Mã hóa file
- Giao thức HTTPS (phần truyền dữ liệu)
- Bảo mật cơ sở dữ liệu

5.2.2. Mật mã khóa công khai (asymmetric cryptography)

Nguyên lý: Dùng 2 khóa:

- Public key (công khai)
- Private key (bí mật)

Một khóa dùng để mã hóa, khóa còn lại dùng để giải mã.

Ví dụ: RSA

Các bước:

- Chọn hai số nguyên tố lớn p, q

- Tính: $N=pq$
 - Tính: $\varphi(N)=(p-1)(q-1)$
 - Chọn e sao cho: $\gcd(e, \varphi(N))=1$
 - Tìm d sao cho: $ed \equiv 1 \pmod{\varphi(N)}$
- Public key: (e, N)
Private key: (d, N)

Mã hóa: $C = M^e \bmod N$

Giải mã: $M = C^d \bmod N$

Ưu điểm

- Không cần chia sẻ khóa bí mật
- Phù hợp xác thực và chữ ký số

Nhược điểm:

- Chậm hơn khóa đối xứng
- Phụ thuộc vào độ khó phân tích số

5.3. Hàm băm (hash function)

Định nghĩa Hàm băm là hàm:

- Nhận đầu vào bất kỳ độ dài
- Trả về chuỗi cố định độ dài: $H(M)=h$

Tính chất

- Một chiều (không đảo ngược được)
- Chống va chạm
- Nhạy với thay đổi nhỏ

Ứng dụng

- Lưu mật khẩu
- Kiểm tra tính toàn vẹn file
- Blockchain
- Xây dựng chữ ký số

5.4. Chữ ký số (digital signature)

Nguyên lý: Kết hợp:

- Hàm băm
- Mật mã khóa công khai

Quy trình:

- Tạo hash của thông điệp
- Mã hóa hash bằng private key
- Người nhận giải mã bằng public key
- So sánh hash

⇒ Đảm bảo:

- Xác thực người gửi
- Toàn vẹn dữ liệu
- Không thể chối bỏ

⇒ Tổng quan mối liên hệ toàn bộ

Thành phần	Vai trò
Số nguyên tố	Nền tảng RSA
Modulo	Không gian tính toán
Khóa đối xứng	Mã hóa nhanh
Khóa công khai	Trao đổi khóa, xác thực
Hàm băm	Toàn vẹn
Chữ ký số	Xác thực và không chối bỏ

5.5. Ứng dụng thực tế
HTTPS

Ngân hàng điện tử

Ví điện tử

Blockchain

Giao dịch điện tử

Lưu mật khẩu

PHẦN 2 - CHỦ ĐỀ NGHIÊN CỨU SÂU: NHỮNG TÌM HIỂU NGHIÊN CỨU KỸ VÀ SÂU VỀ 1 CHỦ ĐỀ, VẤN ĐỀ, BÀI TOÁN. CÓ THỂ CHỌN 1 TRONG HAI HƯỚNG SAU:

Chọn một bài báo trong danh sách được cung cấp và thực hiện các nhiệm vụ sau: Đọc hiểu các nội dung bài báo; Triển khai thuật toán được cung cấp, gợi ý trong bài báo. (xem danh sách các bài báo cung cấp ở mục 4 dưới đây)

BÀI TOÁN XẾP LỊCH THI BẰNG TÔ MÀU ĐỒ THI

Việc xếp lịch thi cho các trường đại học cũng là một bài toán khó. Do có số lượng sinh viên và môn học lớn dẫn đến việc sắp xếp lịch thi trở nên rối. Các lịch thi được sắp xếp không phải lúc nào cũng là tối ưu cho tất cả sinh viên và giảng viên. Còn việc lập lịch thi thủ công thì lại rất tốn nhân lực và thời gian. Vì vậy cần có thuật toán hỗ trợ xếp lịch thi để tận dụng được tài nguyên một cách hợp lý.

Khi lập lịch thi sẽ phải tuân thủ các ràng buộc để có thể tạo ra một lịch thi thuận lợi cho sinh viên. Trong đó các ràng buộc cứng là các ràng buộc không được vi phạm, còn các ràng buộc mềm thì không bắt buộc phải thỏa mãn nhưng sẽ làm bài toán trở nên thực tế hơn.

Chuyển đổi sang đồ thị:

- **Đỉnh (V):** Mỗi môn học là một đỉnh của đồ thị.
- **Cạnh (E):** Nối giữa hai môn thi khi có sinh viên đăng ký học cả hai môn đó (Xung đột)
- **Màu sắc:** Mỗi màu đại diện cho một ca thi (buổi thi) khác nhau.
- **Mục tiêu:** Dùng tô màu đồ thị để sắp xếp được lịch thi cho các môn sao cho thỏa mãn các ràng buộc của đề bài

Thuật toán áp dụng: 1- Thuật toán tham lam + heuristic và 2-Thuật toán DSATUR (Degree of Saturation)

I. Thuật toán tham lam + heuristic

Thuật toán tham lam sẽ ưu tiên sắp xếp các đỉnh có độ xung đột lớn nhất để tô màu trước.

Các bước thực hiện

B1: Xử lý số liệu sinh viên và môn học

- Đọc các bảng môn học, và sinh viên đăng ký môn học
- Xử lý số liệu đã đọc và tổng hợp lại từng môn học có những sinh viên nào học

B2: Xây dựng ma trận kề

- Từ số liệu môn học và sinh viên đã xử lý ở trên, ta tính toán từng môn có bao nhiêu sinh viên đăng ký trùng bằng công thức:

$$P12 = (T12/L1) * 100$$

Trong đó:

P12: Tỷ lệ sinh viên học trùng giữa môn học 1 và 2

T12: Số lượng sinh viên học trùng giữa 2 môn học

L1: Tổng số sinh viên học môn thứ 1

- Sau khi tính toán được tỉ lệ sinh viên đăng ký trùng thì ta có thể tìm được ma trận kề và vẽ được đồ thị

B3: Sắp xếp lại các đỉnh của đồ thị

- Vì các môn có càng nhiều xung đột sẽ càng khó để sắp xếp nên ta sẽ sắp xếp lại để có thể ưu tiên xếp các môn này lên các ca thi phía trước.
- Đỉnh nào có càng nhiều bậc thì sẽ được cho lên đầu

B4: Áp dụng tô màu đồ thị và chia ca thi

- Sau khi sắp xếp lại các đỉnh theo thứ tự ưu tiên thì ta sẽ áp dụng thuật toán tô màu đồ thị
- Ta sẽ tô màu đỉnh đầu tiên bằng màu thứ 1
- Duyệt các đỉnh còn lại chưa tô màu, nếu ko nối với đỉnh 1 và chưa tô màu thì sẽ được tô bằng màu thứ 1
- Kiểm tra điều kiện cho các ràng buộc, nếu không thỏa mãn thì chuyển qua đỉnh tiếp theo
- Lặp lại với các màu khác cho đến khi tất cả các đỉnh được tô màu
- Sau khi chạy xong thuật toán thì sẽ cho ra kết quả là các ca thi

Mã giả cho thuật toán tham lam + heuristic

Algorithm: Exam_Scheduling_Coloring

Input: Graph $G = (V, E)$ với V là danh sách môn học, E là các xung đột. (Đây là sau bước tìm được ma trận kề và vẽ được đồ thị)

Output: Phân loại môn học vào các ca thi.

1. Sắp xếp V theo thứ tự bậc giảm dần (degree).
2. Khởi tạo mảng `result_colors[]` lưu màu của từng đỉnh, mặc định = -1.
3. Khởi tạo biến `current_color = 0` là màu hiện tại.
4. Cho mỗi đỉnh u thuộc V :
 - a. Tạo một biến `can_color = True`.
 - b. Duyệt các đỉnh v kề với u :

Nếu v đã được tô màu `current_color` và xung đột với u thì sẽ bỏ qua.

Nếu không thì đỉnh đó sẽ được tô màu `current_color` và gán vào `result_colors`
 - c. Kiểm tra các ràng buộc, nếu không thỏa mãn thì bỏ qua
5. Trả về mảng `result_colors`.

II. Thuật toán DSATUR (Degree of Saturation)

Thuật toán DSATUR sẽ ưu tiên tô các màu khó nhất trước, đó là những đỉnh có nhiều màu khác nhau (bậc bão hòa) đã được sử dụng ở các đỉnh láng giềng xung quanh nó.

Nếu đỉnh có có bậc bão hòa bằng nhau thì sẽ chọn đỉnh có bậc cao nhất thỏa mãn để tô màu.

Các bước thực hiện

B1: Xử lý số liệu sinh viên và môn học

- Đọc các bảng môn học, và sinh viên đăng ký môn học
- Xử lý số liệu đã đọc và tổng hợp lại từng môn học có những sinh viên nào học

B2: Xây dựng ma trận kề

- Từ số liệu môn học và sinh viên đã xử lý ở trên, ta tính toán từng môn có bao nhiêu sinh viên đăng ký trùng bằng công thức:

$$P12 = (T12/L1) * 100$$

Trong đó:

P12: Tỷ lệ sinh viên học trùng giữa môn học 1 và 2

T12: Số lượng sinh viên học trùng giữa 2 môn học

L1: Tổng số sinh viên học môn thứ 1

- Sau khi tính toán được tỷ lệ sinh viên đăng ký trùng thì ta có thể tìm được ma trận kề và vẽ được đồ thị

B3: Áp dụng DSATUR

- Tính bậc và bậc bão hòa của từng đỉnh chưa tô màu
- Chọn đỉnh có bậc bão hòa cao nhất, nếu bậc bằng nhau thì chọn đỉnh có bậc cao nhất để tô màu
- Duyệt các đỉnh còn lại chưa tô màu, nếu ko nối với đỉnh 1 và chưa tô màu thì sẽ được tô bằng màu đó
- Kiểm tra điều kiện cho các ràng buộc, nếu không thỏa mãn thì chuyển qua đỉnh tiếp theo
- Lặp lại với các màu khác cho đến khi tất cả các đỉnh được tô màu
- Sau khi chạy xong thuật toán thì sẽ cho ra kết quả là các ca thi

Mã giả cho thuật toán DSATUR

Algorithm: Dsatur_Exam_Scheduling_Coloring

Input: Graph $G = (V, E)$ với V là danh sách môn học, E là các xung đột. (Đây là sau bước tìm được ma trận kề và vẽ được đồ thị)

Output: Phân loại môn học vào các ca thi.

1. Khởi tạo mảng `result_colors[]` lưu màu của từng đỉnh, mặc định = -1.
2. Duyệt qua các đỉnh
 - Khởi tạo biến `max_sat` = -1 là bậc bão hòa lớn nhất
 - Khởi tạo biến `max_deg` = -1 là bậc của đỉnh lớn nhất

- Tính bậc bão hòa và bậc của từng đỉnh, chọn đỉnh có bậc bão hòa lớn nhất, nếu bậc bão hòa bằng nhau thì chọn đỉnh có bậc lớn nhất

3. Khởi tạo biến `current_color = 0` là màu hiện tại.

- Chạy vòng lặp với các đỉnh kề, nếu thỏa mãn điều kiện thì sẽ gán màu hiện tại vào đỉnh đã chọn và ghi vào `result_colors`.
- Nếu các đỉnh kề có xung đột với màu hiện tại thì sẽ cộng `current_color` thêm 1, và chạy tiếp vòng lặp cho đến khi không có xung đột màu với các đỉnh kề.
- Kiểm tra các ràng buộc trước khi gán màu

4. Tiếp tục chạy cho tới khi tất cả các đỉnh được gán màu

5. Trả về mảng `result_colors`.

III. Ứng dụng vào bài toán

Thực nghiệm

1. Dữ liệu đầu vào

Bộ dữ liệu `data.xlsx` gồm 2 sheet

DS HocPhan:

Mã LHP	Mã học phần	Mã môn thi	Số tín chỉ	Hình thức thi
--------	-------------	------------	------------	---------------

DS SV – masked:

MSV	MHP	Tên học phần
-----	-----	--------------

2. Lựa chọn thuật toán

Thuật toán tham lam kết hợp với sắp xếp theo bậc (Degree heuristic)

Bài toán xếp lịch thi là bài toán tô màu đồ thị NP-khó. Với số lượng lớn sinh viên và hàng trăm môn học các thuật toán duyệt thủ công sẽ bị quá tải thuật toán Greedy cho phép tìm ra lời giải tốt trong thời gian ngắn. Coi mỗi môn học là một đỉnh, mỗi xung đột sinh viên là một cạnh trong đồ thị.

3. Các ràng buộc

Ràng buộc cứng	Ràng buộc mềm
Một sinh viên không thi 2 học phần cùng ca	Một sinh viên không thi 2 ca liên tiếp
Một phòng không xếp 2 môn cùng ca	Một sinh viên không thi quá 2 ca/ngày
Một học phần có nhiều lớp học phần thì xếp chung 1 lịch thi	Môn 3 tín chỉ ưu tiên buổi sáng hoặc ca đầu buổi chiều
Không xếp quá giới hạn sinh viên trong mỗi phòng thi	Tránh cho 1 SV thi 3 ngày liên tục
Không tổ chức môn thi vào thứ 7 và chủ nhật (tổ chức thi trong 20 ngày)	Các môn cùng khoa không thi cùng ca

4. Các bước thực hiện

4.1. Tiền xử lý dữ liệu

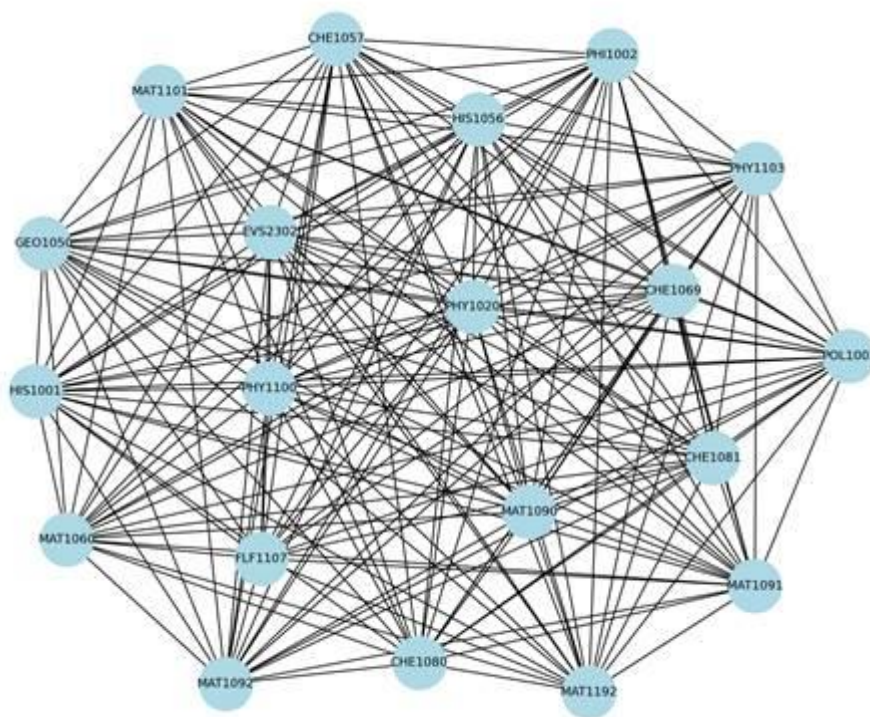
Đọc các danh sách thô từ Excel, thực hiện loại bỏ các dòng trống và chuyển tất cả chữ cái về dạng in hoa và xóa các khoảng trắng dư thừa, tiến hành tách mã học phần gốc từ các mã lớp học phần chi tiết.

Mục đích: Đảm bảo dữ liệu đồng nhất. Việc đưa về mã học phần gốc giúp gom tất cả sinh viên thi cùng một môn vào một nhóm duy nhất, tránh việc chia nhỏ lịch thi cho từng lớp lẻ, giúp quản lý tập trung và dễ ra đề thi chung.

4.2. Xây dựng đồ thị xung đột

Lấy toàn bộ danh sách sinh viên để tìm ra những cặp môn học có chung sinh viên đăng ký. Từ đó, tính toán Bậc xung đột tức là đếm xem một môn học đang có chung sinh viên với bao nhiêu môn học khác.

Mục tiêu: Xác định độ khó của từng môn. Những môn có số lượng sinh viên trùng với nhiều môn khác sẽ được ưu tiên xếp lịch trước để đảm bảo sinh viên không thi 2 môn trong cùng một ca.



Đồ thị xung đột của 15 môn có xung đột cao nhất

4.3. Sắp xếp lịch thi và tối ưu hóa ca thi

Dựa trên bậc đồ thị xung đột tính ở bước trên lần lượt đặt các môn học vào các ca thi trống. Tại mỗi ca sẽ thực hiện các phép thử để kiểm tra xem có vi phạm quy định (trùng môn, trùng khoa) hay không. Ngoài ra còn tính toán các điểm phạt cho những ca thi gây bất tiện cho sinh viên như thi 2 ca liên tiếp trong ngày hoặc thi 2 ngày liên tục.

Mục đích: Tìm ra thời điểm thi phù hợp nhất cho từng môn học. Mục tiêu là vừa đảm bảo không có xung đột, vừa tạo ra một lịch thi tốt nhất cho sinh viên, giúp sinh viên có đủ thời gian nghỉ giữa các môn.

4.4. Phân bổ phòng thi theo quy mô và sức chứa

Sau khi thuật toán xác định được ca thi tối ưu cho từng học phần sẽ tiến hành phân phòng thi dựa trên tổng số lượng sinh viên đăng ký thực tế. Với tối đa 30 sinh viên/phòng số lượng phòng thi cần thiết cho mỗi môn được tính toán tự động nhằm đảm bảo không có phòng thi nào bị vượt quá giới hạn. Điểm nổi bật

trong việc xử lý xếp phòng là áp dụng chia đều sinh viên mỗi phòng thay vì phương pháp lấp đầy thông thường.

Cụ thể sẽ dàn đều tổng số sinh viên vào tất cả các phòng đã chọn để triệt tiêu sự chênh lệch mật độ giữa các phòng thi. Ví dụ, với một học phần có 73 sinh viên, thay vì xếp hai phòng 30 người và một phòng chỉ có 13 người, thuật toán sẽ tự động điều phối thành các phòng có số 25, 24 và 24 sinh viên. Phương pháp này giúp tối ưu hóa không gian, tránh tình trạng cục bộ có phòng quá chật trong khi phòng khác lại quá vắng, mặt khác việc duy trì số lượng sinh viên ổn định giúp cán bộ coi thi dễ dàng quản lý và đảm bảo tính công bằng tuyệt đối cho mọi thí sinh trong suốt quá trình làm bài.

4.5. *Đánh giá độ phức tạp*

N: Số lượng học phần (môn thi).

S: Số lượng sinh viên trung bình mỗi môn.

T: Số lượng slot thi (Ngày*Ca)

C: Số môn xung đột trung bình

P: Số lượng phòng trong một ca

a) Xử lý xung đột sử dụng 2 vòng lặp lồng nhau để so sánh từng cặp môn học:

```
for i in range(len(hoc_phan)):
    for j in range(i+1, len(hoc_phan)):
        if sv_theo_hp[h1] & sv_theo_hp[h2]:
```

Độ phức tạp: $O(N^2.S)$

Giải thích: duyệt $N(N-1)/2$ cặp môn. Với mỗi cặp, phép giao tập hợp set1 & set2 tốn trung bình $O(S)$.

b) Thuật toán xếp lịch

```
for hp in hoc_phan_sorted:      # Duyệt N môn
    for ngay, ca in slots:      # Duyệt T slots
        hard_conflict(...)      # Kiểm tra xung đột cứng
        soft_penalty(...)      # Tính điểm phạt
```


Độ phức tạp: $O(N.T.S.C)$

Giải thích:

Vòng lặp thử T slot cho N môn. Hàm `soft_penalty` cải tiến tính toán khoảng cách trên toàn bộ trục thời gian cho từng sinh viên S đối với các môn đã xếp C

Đánh giá: Thuật toán tham lam nó không đảm bảo tìm ra phương án tốt nhất toàn cục nhưng chạy rất nhanh so với các thuật toán tối ưu hóa hoàn toàn (như Backtracking $O(T^N)$)

c) Chia phòng thi

Độ phức tạp: $O(T.(P+N_{\text{slot}}))$.

Giải thích: Với mỗi slot T , ta khởi tạo danh sách phòng trống tốn $O(P)$. Sau đó duyệt qua các môn N_{slot} để bốc phòng. Vì số lượng phòng $P=100$ và số môn mỗi ca N_{slot} đều nhỏ, bước này chạy gần như thời gian thực.

Đánh giá hiệu năng thuật toán:

Tính hiệu quả: Thuật toán áp dụng chiến thuật tham lam kết hợp với Hệ thống điểm phạt để tối ưu hóa đồng thời các ràng buộc cứng và mềm. Độ phức tạp tính toán tổng thể được duy trì ở mức đa thức $O(N^2.S)$ trong giai đoạn tiền xử lý và $O(N.T.S.C)$ trong giai đoạn xếp lịch. Điều này cho phép hệ thống xử lý tốt mà tập dữ liệu lớn gồm hàng trăm học phần và hàng vạn sinh viên trong thời gian ngắn

Điểm mạnh: Chiến lược ưu tiên về việc sắp xếp các học phần có bậc xung đột cao xử lý trước giúp giải quyết các nút thắt tài nguyên sớm, từ đó giảm thiểu tối đa tỷ lệ thất bại hoặc phải quay lui.

Hàm phạt: Hàm `soft_penalty` mới không chỉ xét theo ngày mà còn tính toán trên trục thời gian liên tục giúp dàn đều khoảng cách thi và triệt tiêu gần như hoàn toàn các ca thi sát nút.

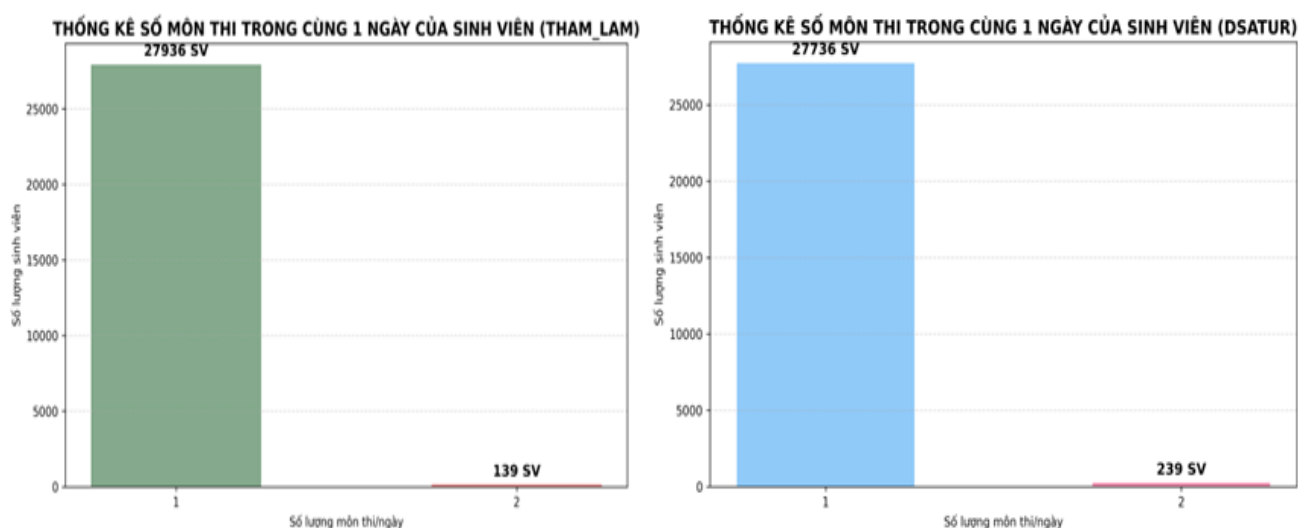
Chia phòng: Chia phòng thực hiện quét danh sách phòng trống một lần duy nhất cho mỗi ca thi, giúp giảm thiểu số vòng lặp thừa và đảm bảo sinh viên được chia đều vào các phòng thi.

4.6. Kết quả

Mã học phần	Tên môn	Số tín chỉ	Thứ	Ngày thi	Giờ thi	Số SV	Phòng thi	Hình thức
0	BIO3202 Động vật học động vật có xương sống	3	Thứ 2	02/02/2026	08:30	50	P041(25), P042(25)	Trắc nghiệm
1	CHE3211 Hoá lý vô cơ	3	Thứ 2	02/02/2026	08:30	7	P044(7)	Khác
2	EVS3307 Tin học ứng dụng trong môi trường	3	Thứ 2	02/02/2026	08:30	5	P046(5)	Khác
3	GLO2210 Xác suất thống kê trong Khoa học Trái đất	3	Thứ 2	02/02/2026	08:30	71	P038(24), P039(24), P040(23)	Viết
4	HMO3609 Dự báo thủy văn biển	3	Thứ 2	02/02/2026	08:30	4	P047(4)	Vấn đáp

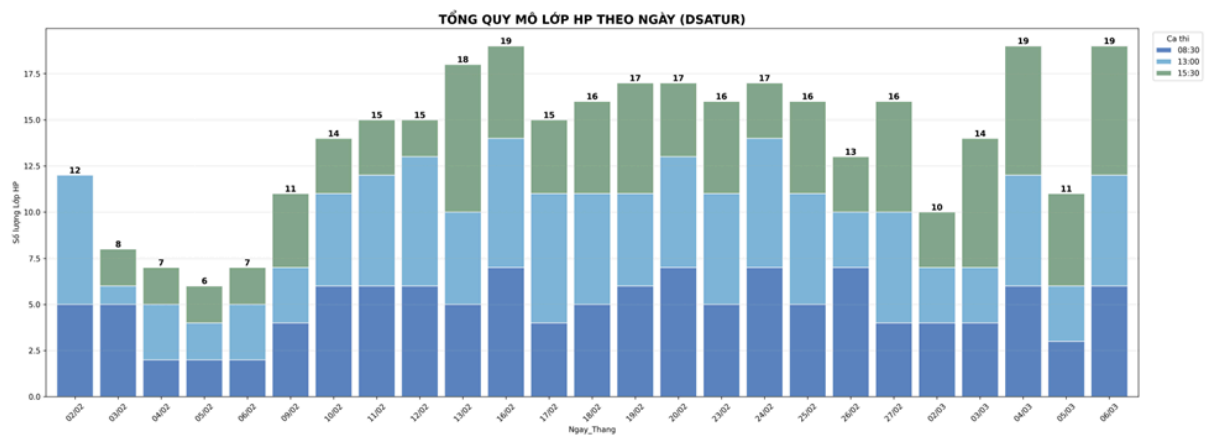
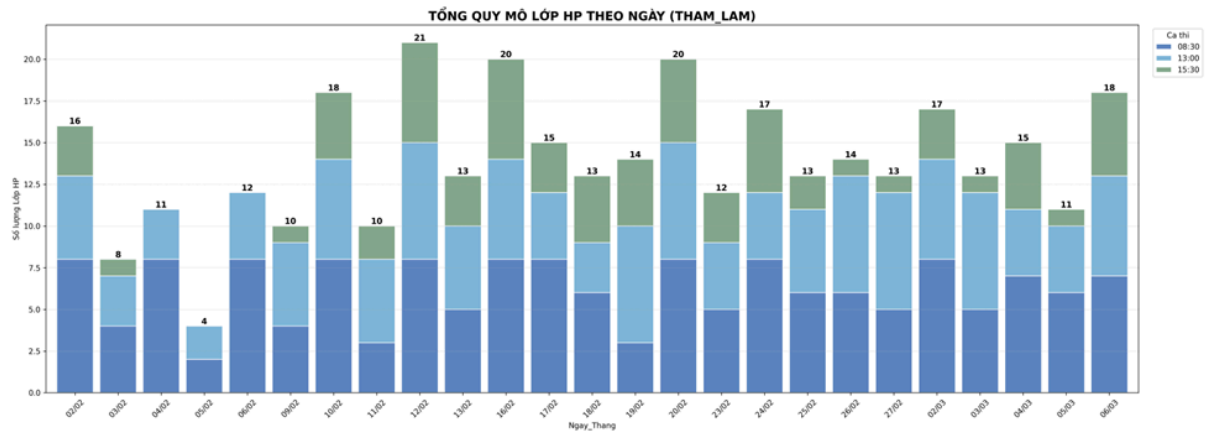
4.7. Thống kê và nhận xét

a) Tải trọng của sinh viên thi



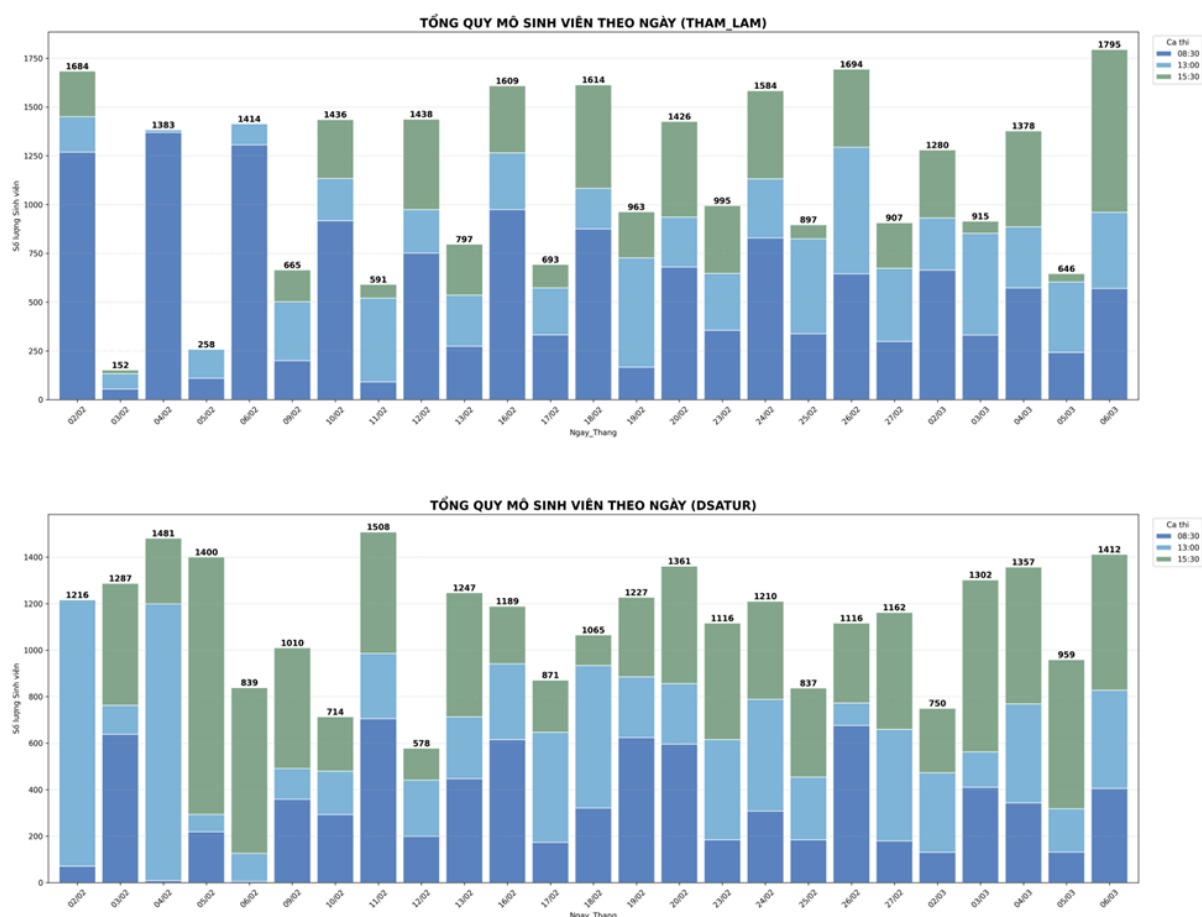
Nhận xét: Kết quả thực nghiệm cho thấy thuật toán Tham lam cho chất lượng phân bố lịch thi theo ngày tốt hơn so với DSATUR. Cụ thể, số lượng sinh viên chỉ phải thi 1 môn trong một ngày ở phương pháp Tham lam cao hơn, trong khi số sinh viên bị dồn 2 môn thi trong cùng một ngày ở DSATUR lớn hơn đáng kể. Điều này cho thấy việc tích hợp ràng buộc mềm trong quá trình xếp lịch của thuật toán Tham lam giúp giảm tình trạng dồn lịch và cải thiện trải nghiệm của sinh viên. Ngược lại, DSATUR chủ yếu tối ưu các ràng buộc cứng của bài toán tô màu đồ thị (tránh xung đột), nên chưa xét đầy đủ đến yếu tố giãn cách lịch thi cho sinh viên, dẫn đến phân bố tải thi theo ngày kém hơn trong bối cảnh bài toán thực tế.

b) Tổng quy mô lớp HP theo ngày



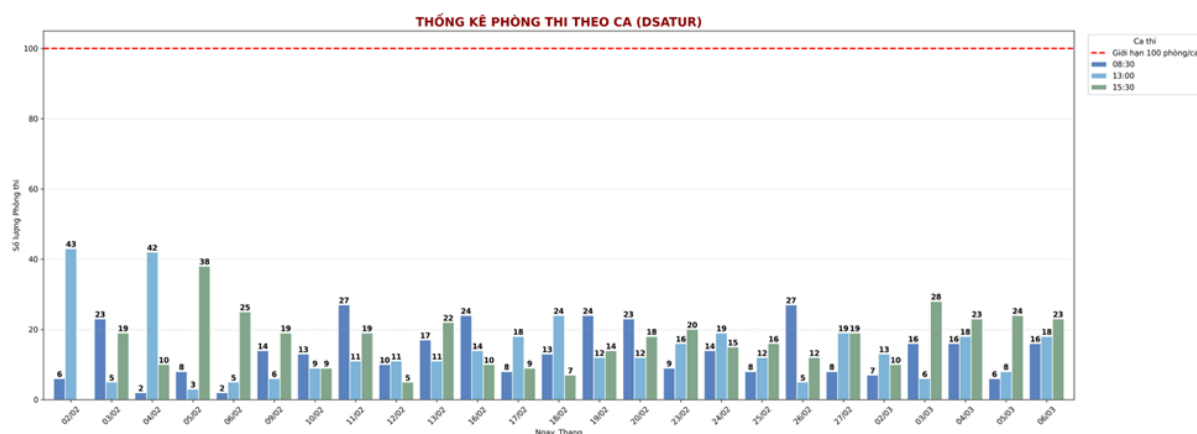
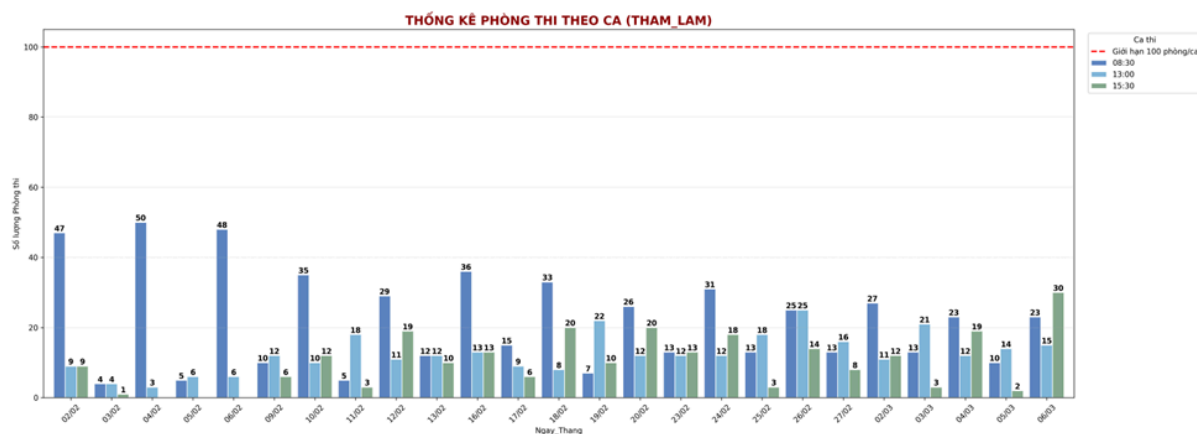
Nhận xét: Hai biểu đồ cho thấy thuật toán **DSATUR** phân bổ số lớp HP theo ngày và theo ca đồng đều hơn, tổng quy mô mỗi ngày dao động ít, ít xuất hiện ngày bị dồn quá nhiều lớp. Trong khi đó, thuật toán **tham lam** có xu hướng tạo ra các ngày “cao điểm” với rất nhiều lớp thi và một số ngày tải thấp, phân bổ không đều. Điều này cho thấy DSATUR giúp cân bằng tải tổ chức tốt hơn, còn tham lam dễ gây quá tải cục bộ theo ngày.

c) Tổng quy mô sinh viên theo ngày



Từ hai biểu đồ có thể thấy **DSATUR** phân bổ số lượng sinh viên theo ngày tương đối đồng đều, tổng quy mô mỗi ngày dao động trong khoảng hẹp và ít xuất hiện ngày bị dồn quá nhiều sinh viên. Ngược lại, **thuật toán tham lam** tạo ra sự chênh lệch rõ rệt giữa các ngày, có những ngày quy mô rất cao và những ngày lại khá thấp, cho thấy hiện tượng dồn tải. Điều này chứng tỏ DSATUR giúp cân bằng tải sinh viên tốt hơn, thuận lợi cho công tác tổ chức thi và giảm áp lực cục bộ so với phương pháp tham lam.

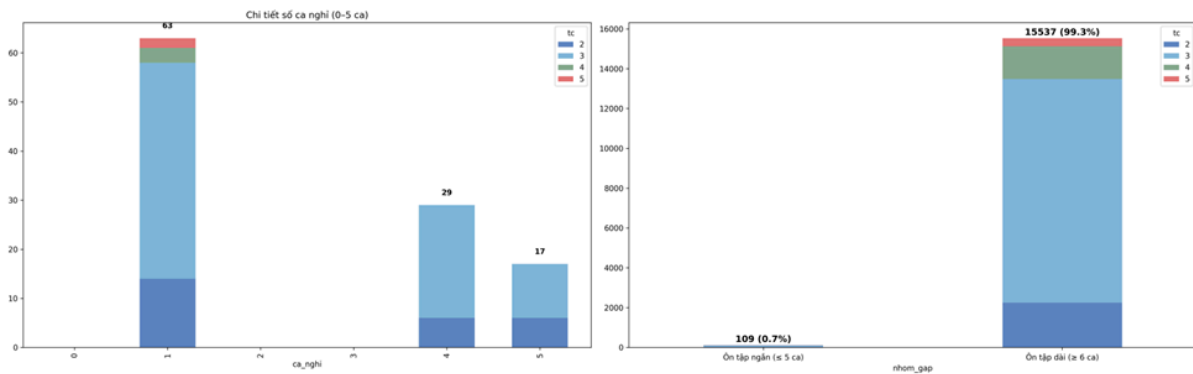
d) Thống kê phòng thi theo ca



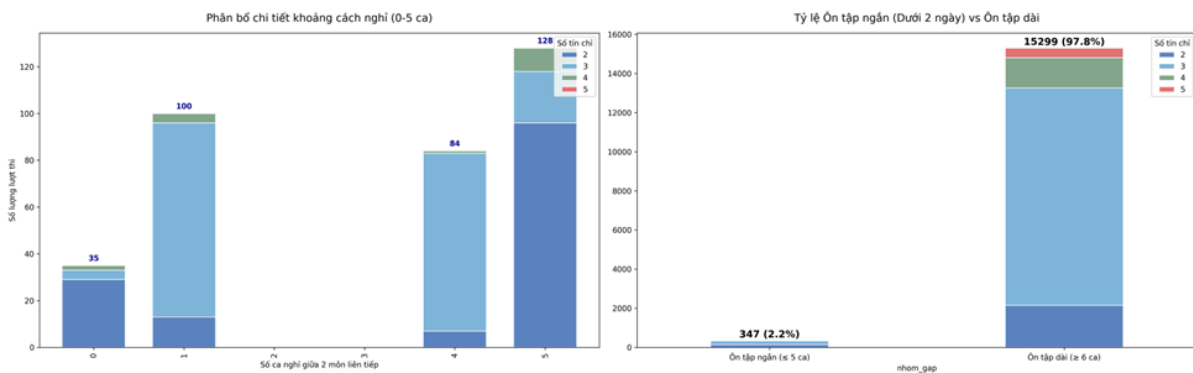
DSATUR phân bổ phòng giữa các ca khá cân bằng hơn theo từng ngày, các ca 08:30–13:00–15:30 không bị dồn lệch quá mạnh; còn thuật toán tham lam có xu hướng dồn nhiều phòng vào ca 08:30, khiến phân bổ lệch rõ rệt dù vẫn không vượt giới hạn 100 phòng/ca. Điều này cho thấy DSATUR tối ưu phân bổ theo cấu trúc xung đột nên “đẹp” hơn về mặt cân bằng ca, trong khi tham lam ưu tiên gán nhanh nên dễ lệch tải giữa các ca.

e) Thời gian ôn tập sinh viên có số môn < 6

PHÂN TÍCH THỜI GIAN ÔN TẬP (THAM_LAM) - NHÓM HỌC ≤ 6 MÔN



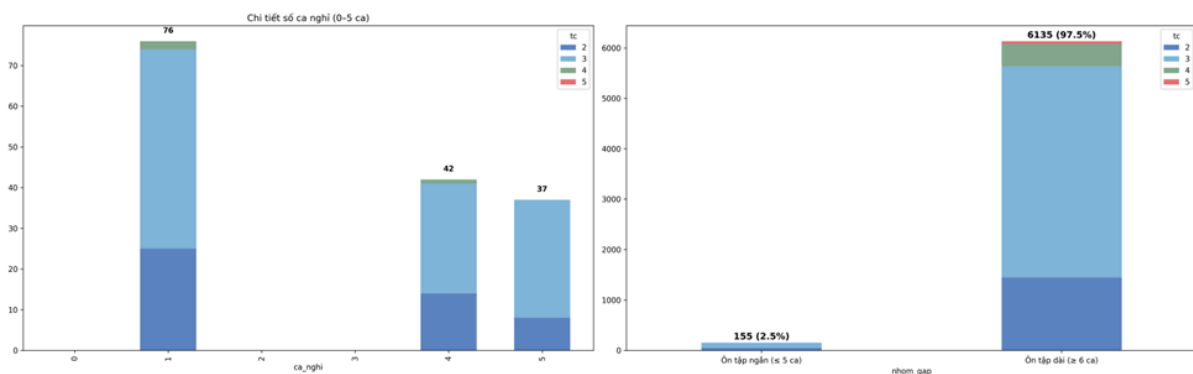
DSATUR: PHÂN TÍCH THỜI GIAN ÔN TẬP - NHÓM ≤ 6 MÔN



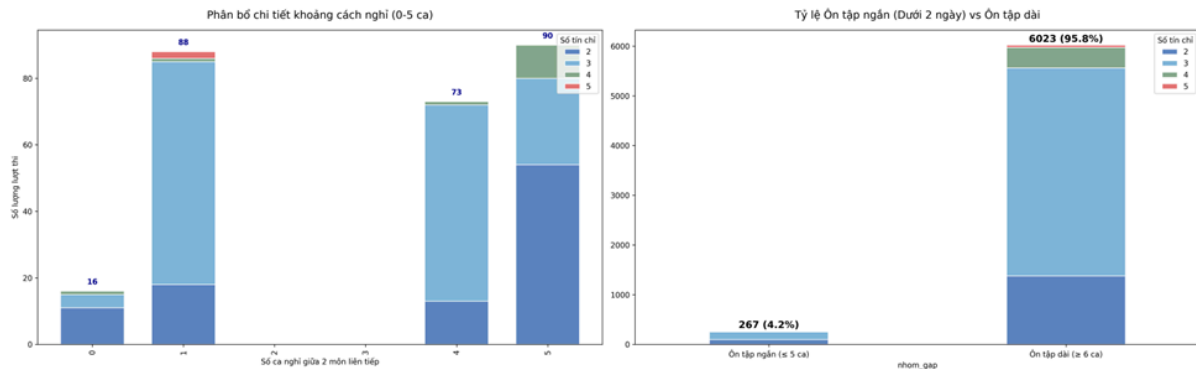
Dựa trên số liệu từ hai biểu đồ thuật toán tham lam cho kết quả tốt hơn DSATUR về mặt giảm áp lực thi liên tiếp cho sinh viên. Tham lam loại bỏ hoàn toàn các trường hợp thi hai ca liên tiếp (0 lượt), trong khi DSATUR vẫn còn 35 lượt. Đồng thời, số lượt có khoảng ôn tập ngắn (≤ 5 ca) ở tham lam chỉ 109 lượt (0,7%), thấp hơn đáng kể so với 347 lượt (2,2%) của DSATUR, còn tỷ lệ khoảng ôn tập dài (≥ 6 ca) tăng từ 97,8% ở DSATUR lên 99,3% ở tham lam.

f) Thời gian ôn tập sinh viên có số môn > 6

PHÂN TÍCH THỜI GIAN ÔN TẬP (THAM_LAM) - NHÓM HỌC > 6 MÔN



DSATUR: PHÂN TÍCH THỜI GIAN ÔN TẬP - NHÓM > 6 MÔN



Ở nhóm sinh viên học trên 6 môn, DSATUR vẫn còn trường hợp thi 2 ca liên tiếp với 16 lượt, trong khi tham lam không còn lượt nào ở mức 0 ca nghỉ. Về phân bố khoảng nghỉ, DSATUR có 88 lượt ở 1 ca nghỉ, 73 lượt ở 4 ca nghỉ và 90 lượt ở 5 ca nghỉ; còn tham lam tương ứng là 76, 42 và 37 lượt, cho thấy tham lam loại bỏ được thi sát ca nhưng phân bố khoảng nghỉ kém đều hơn. Xét tổng thể thời gian ôn tập, DSATUR có 6023 lượt (95,8%) ôn tập dài và 267 lượt (4,2%) ôn tập ngắn, trong khi tham lam có 6135 lượt (97,5%) ôn tập dài và 155 lượt (2,5%), tức tham lam giãn lịch tốt hơn nhưng DSATUR vẫn chấp nhận một số ca thi liên tiếp

g) Thời gian chạy

Thời gian xếp lịch của thuật toán tham lam: 9.0802 giây

Thời gian chạy DSATUR : 26.292303 giây

Thuật toán tham lam chạy nhanh hơn đáng kể so với DSATUR. Cụ thể, tham lam hoàn thành việc xếp lịch trong khoảng 9 giây, trong khi DSATUR mất hơn 26 giây, tức là chậm hơn gần 3 lần. Điều này phù hợp với bản chất hai thuật toán: tham lam duyệt đơn giản theo thứ tự ưu tiên cố định nên chi phí tính toán thấp, còn DSATUR phải liên tục tính bậc bão hòa, so sánh nhiều đỉnh ở mỗi vòng lặp và cập nhật thông tin lân cận nên tốn thời gian hơn. Vì vậy, với bộ dữ liệu lớn hoặc khi cần chạy nhiều lần để thử nghiệm/tham số hóa, tham lam có lợi thế rõ rệt về tốc độ và khả năng mở rộng; ngược lại, DSATUR phù hợp hơn khi chấp nhận đánh đổi thời gian chạy để lấy cấu trúc lịch thi “đẹp” và cân bằng hơn, đặc biệt trong các bài toán có đồ thị xung đột phức tạp.

4.8. Kết luận

Thuật toán tham lam ưu tiên mạnh các ràng buộc mềm liên quan đến trải nghiệm sinh viên, đặc biệt là tránh việc thi hai ca liên tiếp, nên lịch thi tạo ra thân thiện hơn cho người học và đảm bảo có thời gian nghỉ giữa các môn; đổi lại, cách phân bố lịch của tham lam kém linh hoạt hơn và có xu hướng giãn lịch khá xa. Trong khi đó, DSATUR tập trung vào cấu trúc xung đột và tính đẹp của bài toán tô màu, nên lịch thi nhìn cân đối hơn về mặt phân bố ca và ngày, nhưng vẫn chấp nhận tồn tại một số trường hợp sinh viên bị thi sát ca. Về hiệu năng, tham lam tỏ ra vượt trội khi thời gian chạy ngắn hơn nhiều so với DSATUR, cho thấy thuật toán này phù hợp hơn với các bài toán quy mô lớn hoặc khi cần chạy thử nghiệm nhiều lần. Vì vậy, nếu mục tiêu ưu tiên trải nghiệm và sức khỏe sinh viên cùng với tốc độ xử lý, tham lam là lựa chọn phù hợp; còn nếu ưu tiên tính cân đối, gọn gàng của lịch thi và chấp nhận đánh đổi thời gian tính toán, DSATUR vẫn là phương án hợp lý nhưng cần bổ sung thêm cơ chế xử lý ràng buộc mềm để cải thiện trải nghiệm sinh viên.

Tài liệu tham khảo

1. University Exam Scheduling System Using Graph Coloring Algorithm and RFID Technology - Akhan Akbulut and Güray Yılmaz, Member, IACSIT
2. New Graph Coloring Algorithms - Dr. Hussein Al-Omari and Khair Eddin Sabri
3. Slide môn học Toán rời rạc và thuật toán - Nguyễn Thị Hồng Minh